Numerical and simulation techniques in finance

Nicolas Langrené

Beijing Normal-Hong Kong Baptist University

2025 Fall Semester

5 Simulation of multivariate random variables

Multivariate random variables

We now consider the problem of simulating a D-dimensional random vector

$$X_1, X_2, \ldots, X_D$$

where the random variables X_d, $d = 1, \ldots, D$ are possibly dependent, and can have different marginal distributions.

We first consider the simple case where the random variables $X_1, X_2, \ldots, X_D$ are *mutually independent*, and then consider the general case where the random variables $X_1, X_2, \ldots, X_D$ can be *dependent*.

Independent random variables

Two random variables X and Y are **independent** if

$$\mathbb{P}(X \in \mathcal{A}, Y \in \mathcal{B}) = \mathbb{P}(X \in \mathcal{A})\mathbb{P}(Y \in \mathcal{B})$$

for all sets $\mathcal{A}$ and $\mathcal{B}$. Let $f_X(x)$ and $f_Y(y)$ be the **marginal PDFs** of X and Y respectively, and let $f_{X,Y}(x,y)$ be the **joint PDF** of (X,Y). Then X and Y are independent iff

$$f_{X,Y}(x,y) = f_X(x)f_Y(y)$$

for all x, y. Similarly, let $F_X(x)$ and $F_Y(y)$ be the **marginal CDFs** of X and Y respectively, and let $F_{X,Y}(x,y)$ be the **joint CDF** of (X,Y). X and Y are independent iff

$$F_{X,Y}(x,y) = F_X(x)F_Y(y)$$

Mutually independent random variables

D random variables $X_1, X_2, \ldots, X_D$ are **mutually independent** if

$$\mathbb{P}(X_1 \in \mathcal{A}_1, \ldots, X_D \in \mathcal{A}_D) = \mathbb{P}(X_1 \in \mathcal{A}_1) \ldots \mathbb{P}(X_D \in \mathcal{A}_D)$$

for all sets $\mathcal{A}_1, \ldots, \mathcal{A}_D$.

Let $f_{X_d}(x_d)$ be the **marginal PDF** of X_d, $d = 1 \ldots, D$ and let $f_{X_1,\ldots,X_D}(x_1, \ldots, x_D)$ be the **joint PDF** of $X_1, X_2, \ldots, X_D$.
Then $X_1, X_2, \ldots, X_D$ are mutually independent iff

$$f_{X_1,\ldots,X_D}(x_1, \ldots, x_D) = f_{X_1}(x_1) \ldots f_{X_D}(x_D)$$

for all $x_1, \ldots, x_D$. A similar result holds for the CDFs.

Simulate independent random variables

If the D random variables $X_1, X_2, \ldots, X_D$ are mutually independent, then each random variable X_d can be simulated independent according to its marginal distribution, using one of the previously introduced simulation algorithms (inversion method, acceptance-rejection method, etc.)

⚠ The uniform distribution simulations underlying the generation of each simulation with distribution X_d must come from *the same pseudorandom number generator*.

⚠ This straightforward approach *does not work* if the random variables $X_1, X_2, \ldots, X_D$ are dependent.

Correlated random variables

The easiest example of simulation of dependent random variable is the case of two Gaussian variables with prescribed **correlation** ρ.

The correlation (a.k.a. Pearson correlation, or linear correlation) between two random variables X and Y is defined by

$$\rho_{X,Y} = \frac{\mathbb{C}\mathrm{ov}(X,Y)}{\sigma_X \sigma_Y}$$

where $\mathbb{C}\mathrm{ov}(X,Y)$ is the covariance between X and Y, σ_X is the standard deviation of X, and σ_Y is the standard deviation of Y.

Correlated random variables

If two random variables are independent, then $\rho_{X,Y} = 0$. However, the converse is generally not true: two uncorrelated random variables are not necessarily independent.

Classical counterexample: let X be a standard Gaussian variable, and set $Y := X^2$. Note that X and Y are perfectly dependent. However, X and Y are uncorrelated since $\mathbb{E}[XY] = \mathbb{E}[X^3] = 0 = \mathbb{E}[X]\mathbb{E}[Y]$.

✒ Uncorrelatedness is equivalent to independence ONLY in the special case when X and Y follow a joint normal distribution.

✒ Correlation is a good measure of dependence ONLY in the multivariate Gaussian case.

5.1 Simulation of correlated Gaussian variables

Bivariate Gaussian variables

Let $X \sim \mathcal{N}(0,1)$ and $Y \sim \mathcal{N}(0,1)$ be two independent standard Gaussian variables. Then

$$Z := \rho X + \sqrt{1-\rho^2} Y$$

is also a standard Gaussian variable, with correlation ρ between X and Z.

Indeed, Z is a Gaussian variable (as a linear combination of two independent Gaussian variables), has mean $\mathbb{E}[Z] = 0$ and variance $\mathbb{V}\mathrm{ar}[Z] = \rho^2 \mathbb{V}\mathrm{ar}[X] + (1-\rho^2)\mathbb{V}\mathrm{ar}[Y] = 1$, and therefore is also a standard Gaussian variable $Z \sim \mathcal{N}(0,1)$. Moreover, its correlation with X is

$$\rho_{X,Z} = \mathbb{E}[XZ] = \rho \mathbb{E}[X^2] + \sqrt{1-\rho^2}\mathbb{E}[X]\mathbb{E}[Y] = \rho$$

Multivariate Gaussian variables

The multivariate Gaussian distribution $\mathcal{N}(\mu, \mathbf{\Sigma})$, where $\mu \in \mathbb{R}^D$ is a location vector and $\mathbf{\Sigma} \in \mathbb{R}^{D \times D}$ is a covariance matrix, is defined by the PDF

$$f(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^D \det(\mathbf{\Sigma})}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^\top \mathbf{\Sigma}^{-1} (\mathbf{x} - \mu) \right)$$

Using this new notation $\mathcal{N}(\mu, \mathbf{\Sigma})$, the bivariate Gaussian vector $[X, Z]^\top$ described in the previous page has mean $\mu = [0, 0]^\top$ and covariance

$$\mathbf{\Sigma} = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$$

Simulate multivariate Gaussian variables

We need a method to simulate any multivariate Gaussian distribution $\mathcal{N}(\mu, \mathbf{\Sigma})$ for any dimension D, any mean vector μ and positive-definite covariance matrix $\mathbf{\Sigma}$.

The main result to use is the following: if the covariance matrix can be decomposed as

$$\mathbf{\Sigma} = \mathbf{A}\mathbf{A}^\top$$

Then if $\mathbf{N} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is a standard multivariate Gaussian vector, then

$$\mathbf{X} = \mu + \mathbf{A}\mathbf{N} \sim \mathcal{N}(\mu, \mathbf{\Sigma})$$

Simulate bivariate Gaussian variables

$$\mathbf{X} = \mu + \mathbf{A}\mathbf{N} \sim \mathcal{N}(\mu, \mathbf{\Sigma})$$

We check this formula in the bivariate case. In the example discussed previously, we simulated two correlated Gaussian variables

$$\begin{bmatrix} X_1 \\ X_2 \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix} \right)$$

from two independent standard Gaussian variables

$$\begin{bmatrix} N_1 \\ N_2 \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right)$$

Simulate bivariate Gaussian variables

One can check that the square root $\mathbf{A}$ of the covariance matrix

$$\mathbf{\Sigma} = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$$

is given by

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ \rho & \sqrt{1-\rho^2} \end{bmatrix}$$

Indeed, one can verify that $\mathbf{\Sigma} = \mathbf{A}\mathbf{A}^\top$

Simulate bivariate Gaussian variables

Multiplying $\mathbf{A}$ by a vector of independent standard Gaussian variables yields

$$\mathbf{A}\begin{bmatrix} N_1 \\ N_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \rho & \sqrt{1-\rho^2} \end{bmatrix}\begin{bmatrix} N_1 \\ N_2 \end{bmatrix} = \begin{bmatrix} N_1 \\ \rho N_1 + \sqrt{1-\rho^2} N_2 \end{bmatrix} = \begin{bmatrix} X_1 \\ X_2 \end{bmatrix}$$

This verifies that multiplying $[N_1, N_2]^\top \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ by the matrix $\mathbf{A}$ produces a bivariate Gaussian vector

$$\begin{bmatrix} X_1 \\ X_2 \end{bmatrix} \sim \mathcal{N}\left(\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \mathbf{\Sigma} = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}\right)$$

where $\mathbf{\Sigma} = \mathbf{A}\mathbf{A}^\top$

Simulate bivariate Gaussian variables

More generally, we can simulate a bivariate Gaussian vector with general mean μ and general covariance matrix $\mathbf{\Sigma}$. A general bivariate covariance matrix can be written as

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_X^2 & \rho\sigma_X\sigma_Y \\ \rho\sigma_X\sigma_Y & \sigma_Y^2 \end{bmatrix}$$

The square-root matrix $\mathbf{A}$ is given by

$$\begin{bmatrix} \sigma_X & 0 \\ \rho\sigma_Y & \sigma_Y\sqrt{1-\rho^2} \end{bmatrix}$$

Simulate bivariate Gaussian variables

Let $[N_1, N_2]^\top \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Then

$$\begin{bmatrix} X_1 \\ X_2 \end{bmatrix} := \mu + \mathbf{A} \begin{bmatrix} N_1 \\ N_2 \end{bmatrix} = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix} + \begin{bmatrix} \sigma_1 & 0 \\ \rho\sigma_2 & \sigma_2\sqrt{1-\rho^2} \end{bmatrix} \begin{bmatrix} N_1 \\ N_2 \end{bmatrix} = \begin{bmatrix} \mu_1 + \sigma_1 N_1 \\ \mu_2 + \sigma_2(\rho N_1 + \sqrt{1-\rho^2} N_2) \end{bmatrix}$$

$[X_1, X_2]^\top \sim \mathcal{N}(\mu, \mathbf{\Sigma})$ has the desired distribution. In particular

$$\begin{aligned} X_1 &\sim \mathcal{N}(\mu_1, \sigma_1^2) \\ X_2 &\sim \mathcal{N}(\mu_2, \sigma_2^2) \\ \mathbb{C}\mathrm{orr}(X_1, X_2) &= \rho \end{aligned}$$

Simulate multivariate Gaussian variables

Let $\mathbf{N} = [N_1, N_2, \ldots, N_D]^\top \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. These independent Gaussian variables can be simulated, for example, from the Box-Muller algorithm.

In order to simulate $\mathbf{X} = [X_1, X_2, \ldots, X_D]^\top \sim \mathcal{N}(\mu, \mathbf{\Sigma})$, the only difficulty is to compute the square-root matrix $\mathbf{A}$ of $\mathbf{\Sigma} = \mathbf{A}\mathbf{A}^\top$. This can be achieved using the **Cholesky decomposition**. It is a numerically stable algorithm, with computational complexity $\mathcal{O}(D^3)$, and is twice faster than the LU decomposition.

Then, computing $\mathbf{X} = \mu + \mathbf{A}\mathbf{N}$ provides simulations with the desired distribution $\mathbf{X} \sim \mathcal{N}(\mu, \mathbf{\Sigma})$

Bivariate Gaussian simulation with SciPy

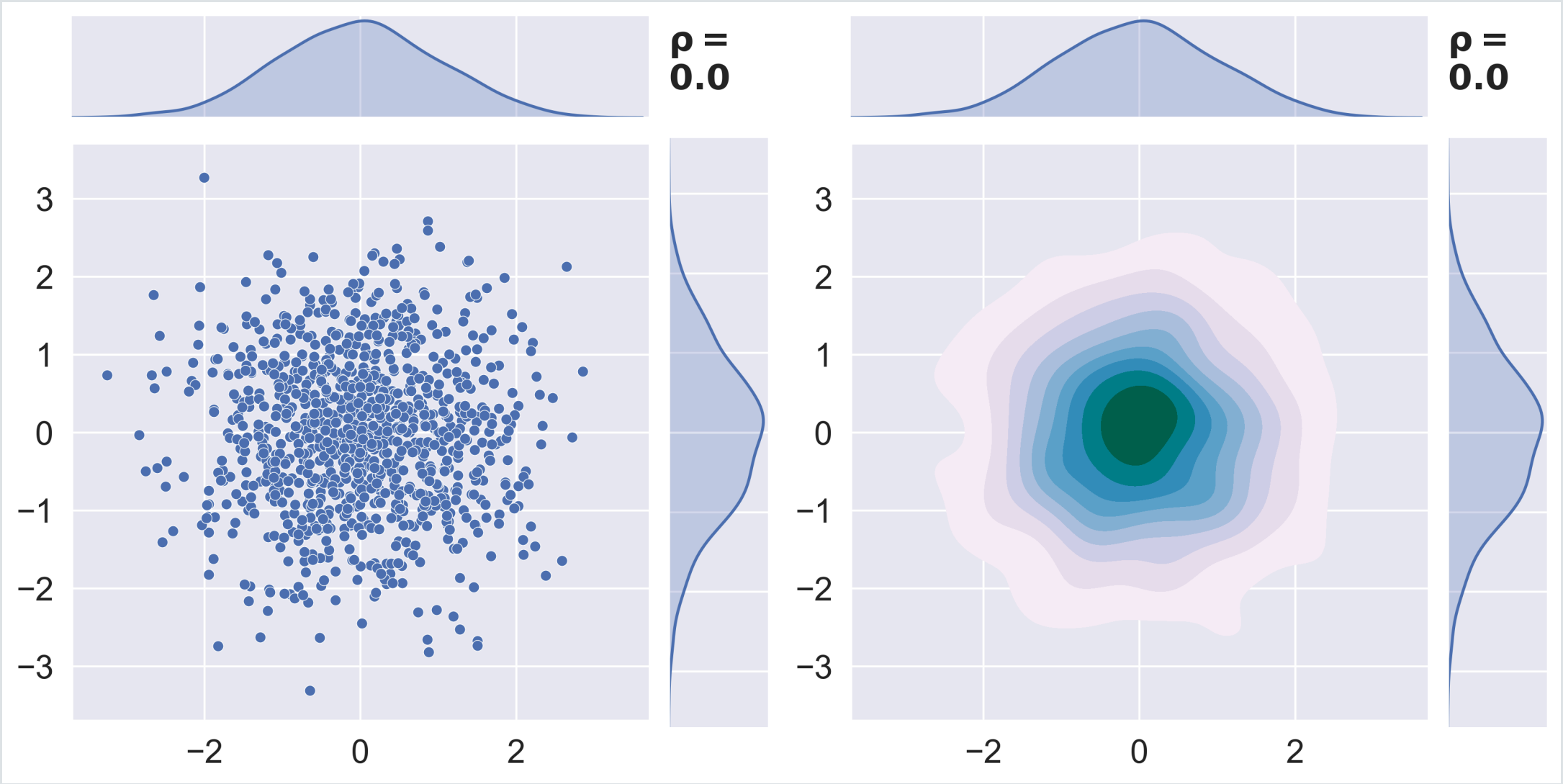
https://numpy.org/doc/stable/reference/generated/numpy.linalg.cholesky.html

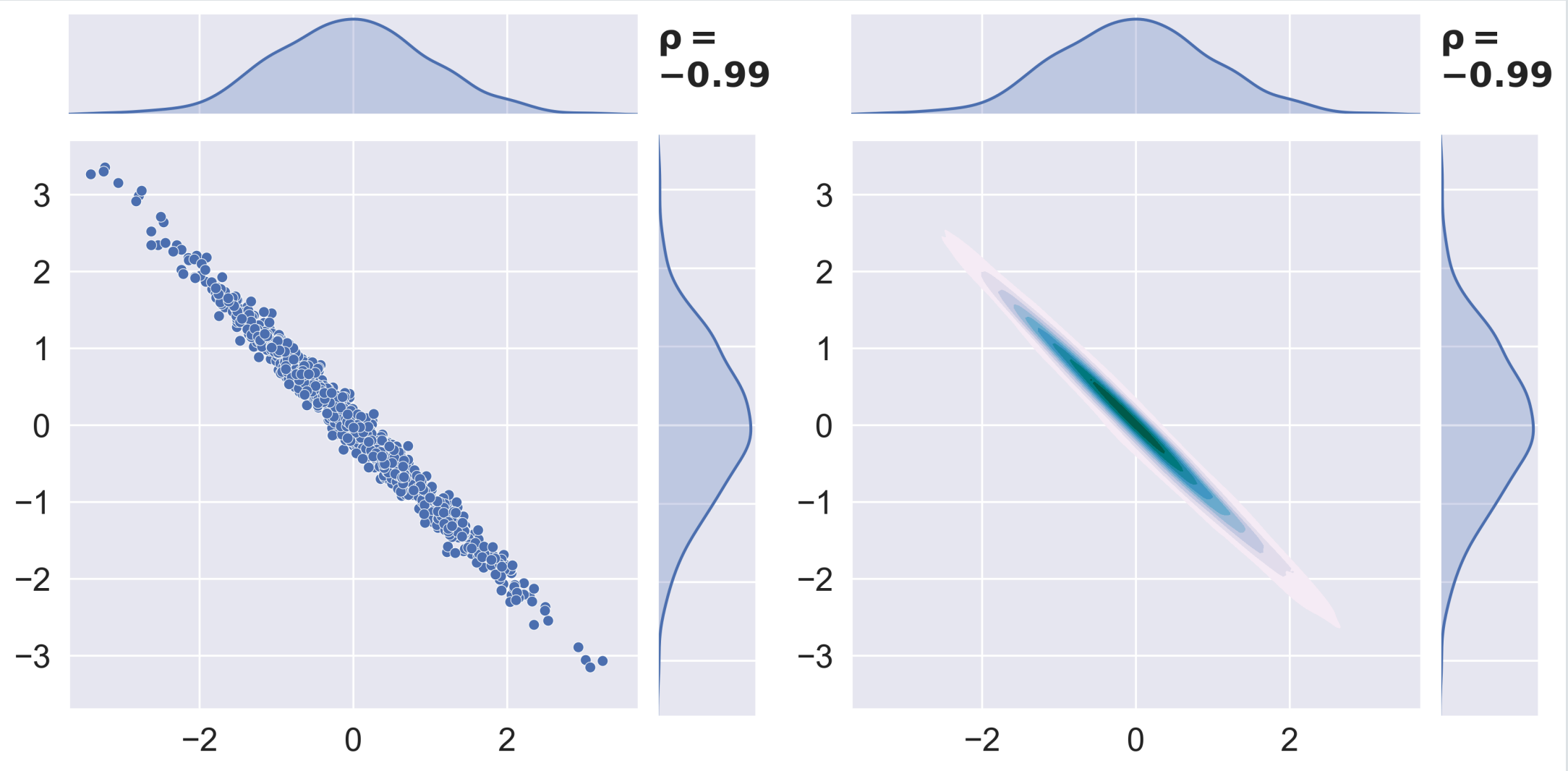
```
import numpy as np; from scipy.stats import norm
mu = np.array([[10],
               [30]])
sigma1, sigma2, rho = 0.4, 0.3, 0.35
S = np.array([[sigma1*sigma1    ,rho*sigma1*sigma2] ,
              [rho*sigma1*sigma2,   sigma2*sigma2]])
A = np.linalg.cholesky(S)
np.allclose(A@A.T,S)
## True
N = norm.rvs(size=[2,1000])
X = mu + A@N
( np.mean(X[0,:]) , np.std(X[0,:]) )
## (9.995389426873793, 0.388471578797909 5)
( np.mean(X[1,:]) , np.std(X[1,:]) )
## (29.997948091741026, 0.297542772123409 7)
np.corrcoef(X[0,:],X[1,:])[0,1]
## 0.3254165591725856 6
```

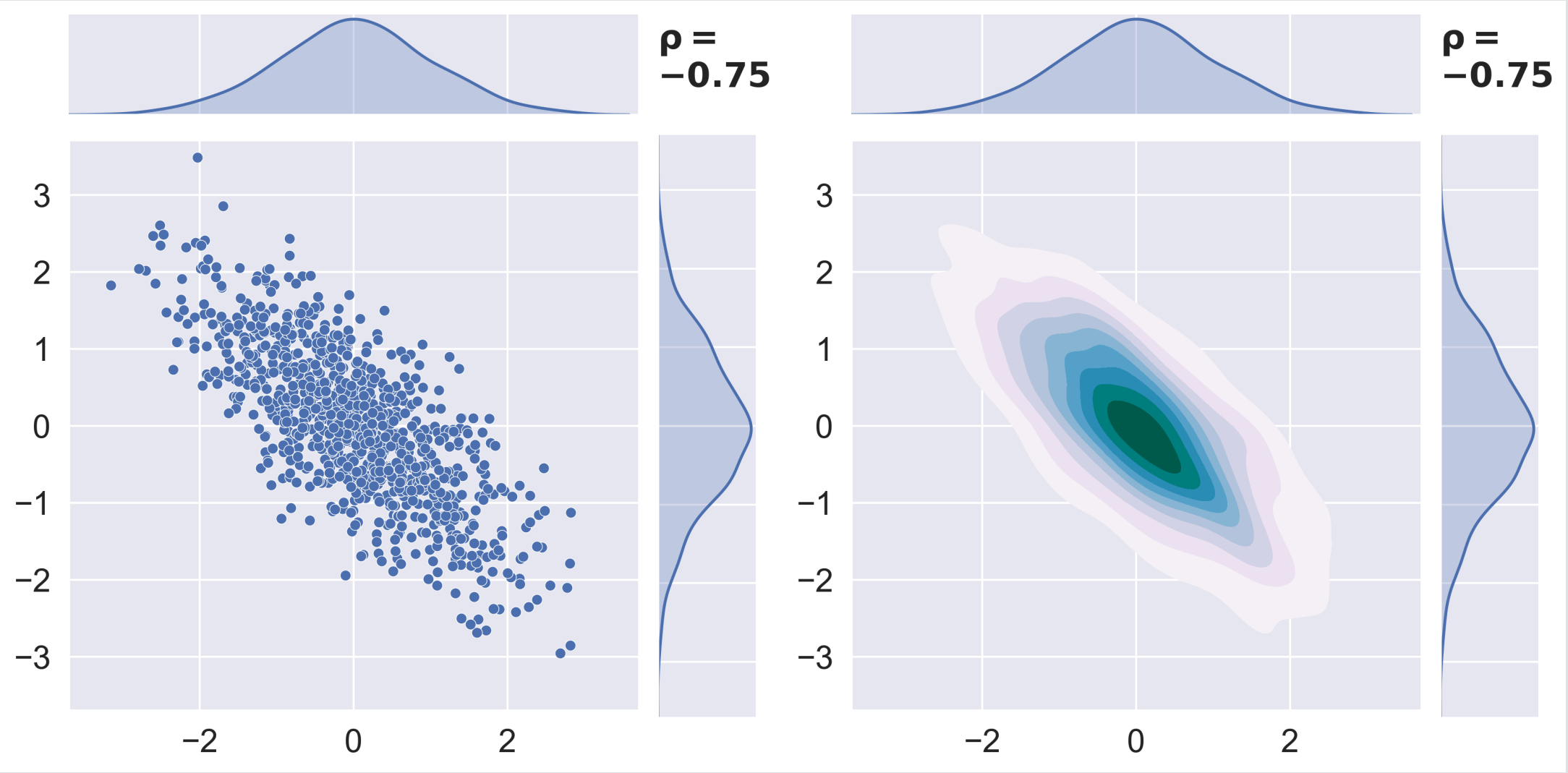
Bivariate Gaussian simulation with SciPy

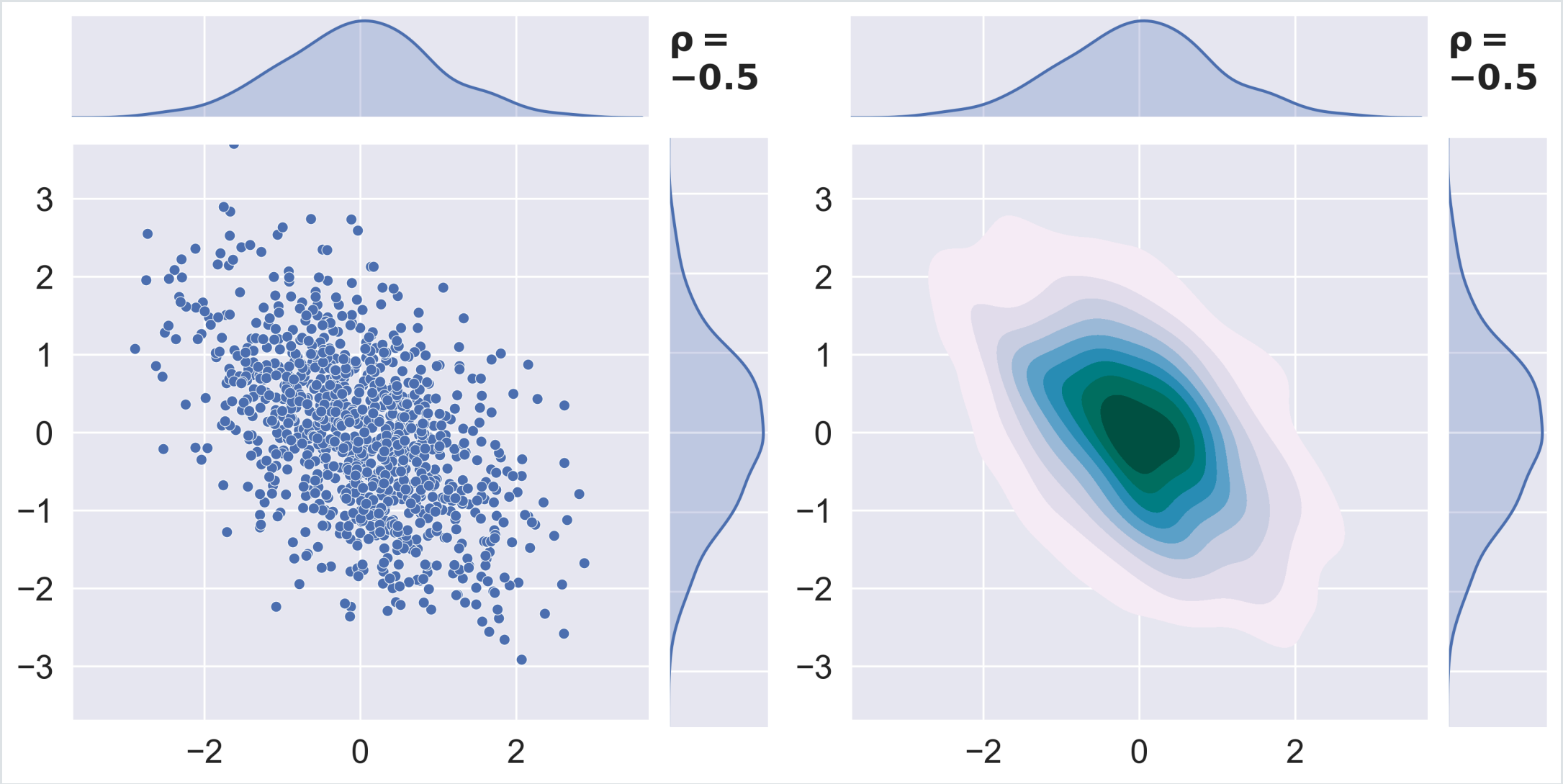
https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.multivariate_normal.html

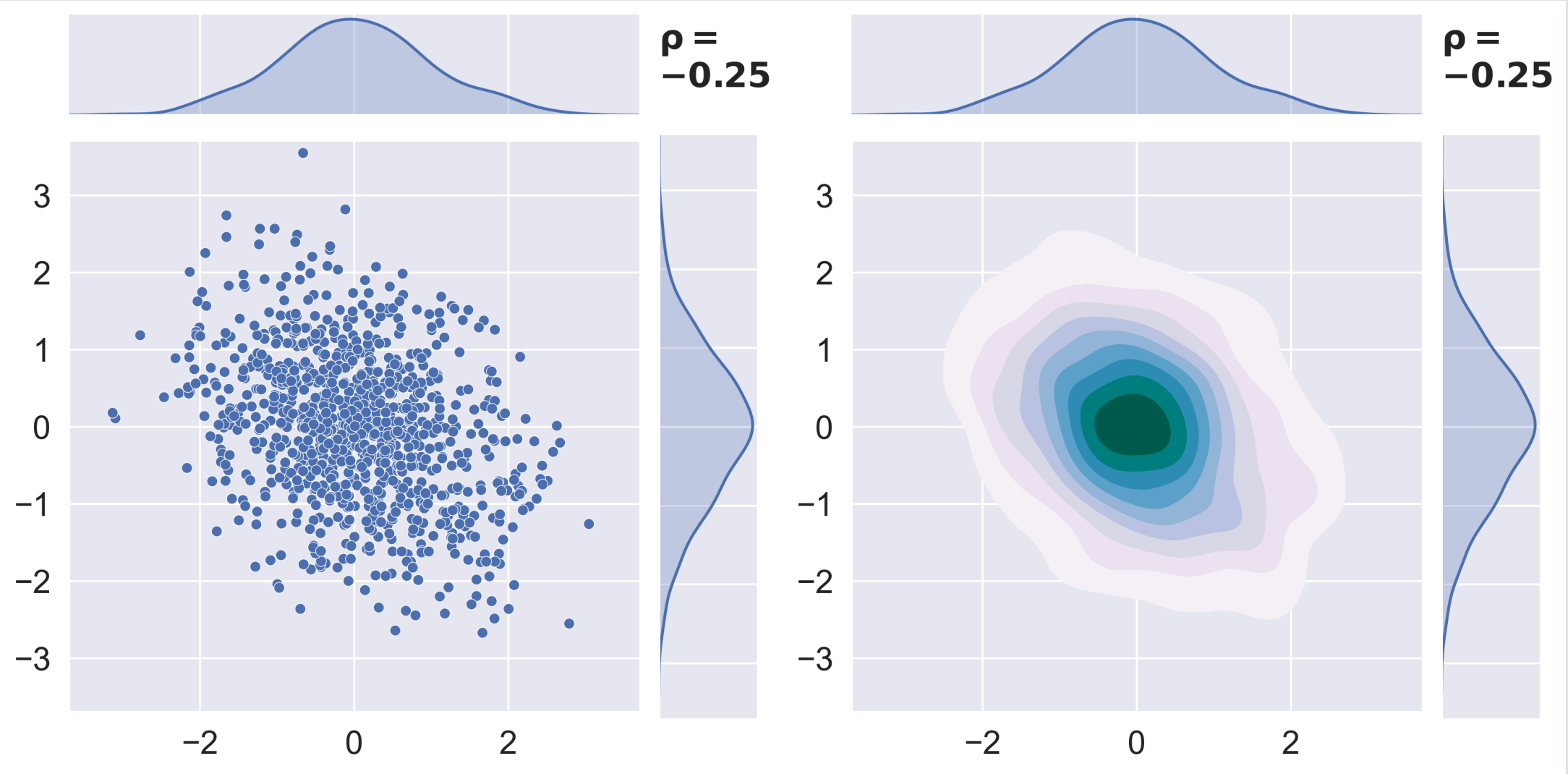
```
import numpy as np
from scipy.stats import multivariate_normal
mu = np.array([10,30])
sigma1, sigma2, rho = 0.4, 0.3, 0.35
S = np.array([[sigma1*sigma1    ,rho*sigma1*sigma2] ,
              [rho*sigma1*sigma2,   sigma2*sigma2]])
X = multivariate_normal.rvs(mean=mu,cov=S,size=1000)
np.shape(X)
## (1000, 2)
np.mean(X[:,0]) , np.std(X[:,0])
## (9.988710128001653, 0.3987211960236535)
np.mean(X[:,1]) , np.std(X[:,1])
## (29.99889404520276, 0.29732638849254 1)
np.corrcoef(X[:,0],X[:,1])[0,1]
## 0.3390915194394701
```

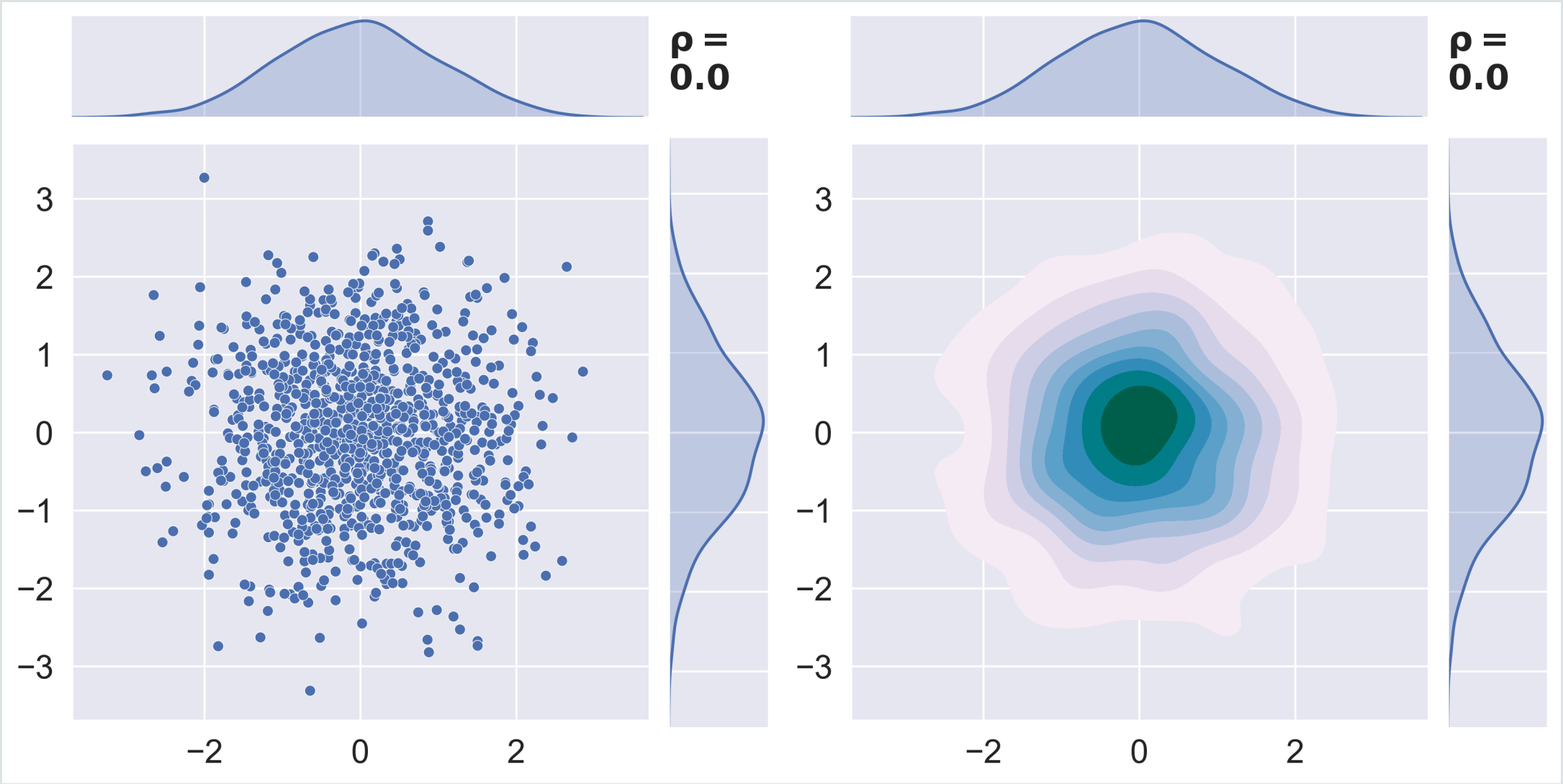

ρ = 0.0
ρ = 0.0
3
2
1
0
−1
−2
−3
−2
0
2
3
2
1
0
−1
−2
−3
−2
0
2

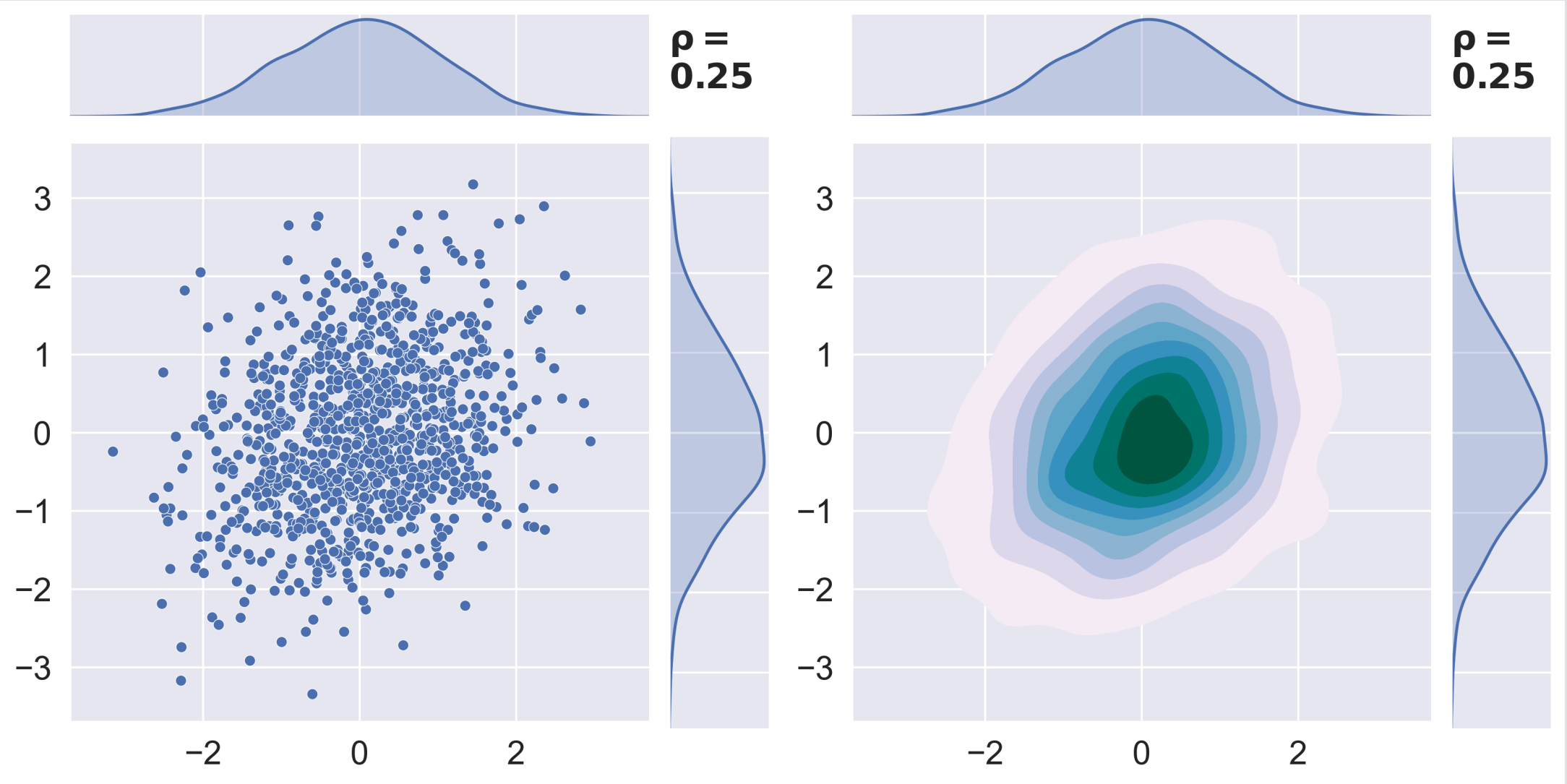

ρ =
−0.99
ρ =
−0.99
3
2
1
0
−1
−2
−3
−2
0
2
3
2
1
0
−1
−2
−3
−2
0
2

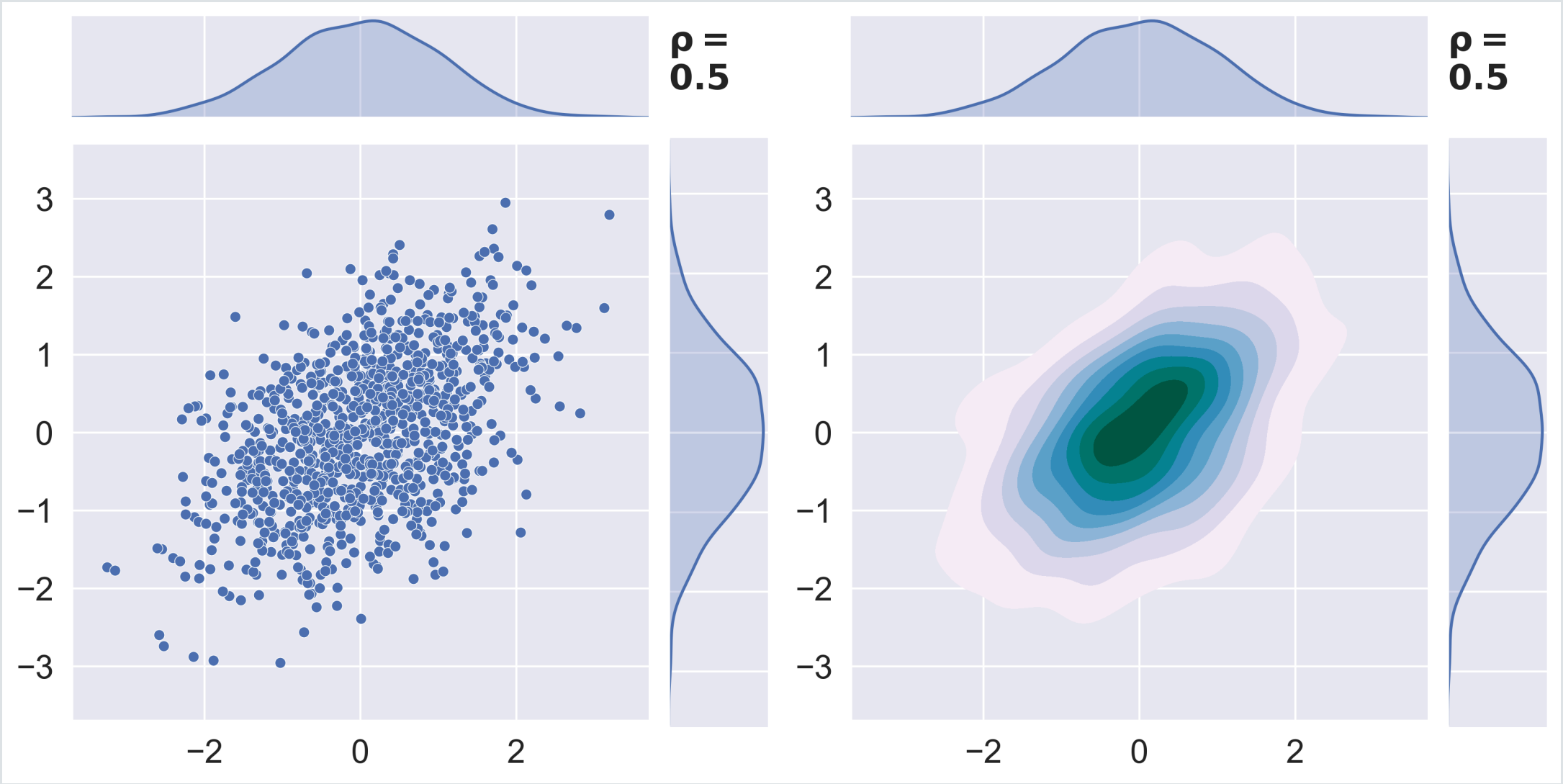

ρ =
−0.75
ρ =
−0.75
3
2
1
0
−1
−2
−3
−2
0
2
3
2
1
0
−1
−2
−3
−2
0
2

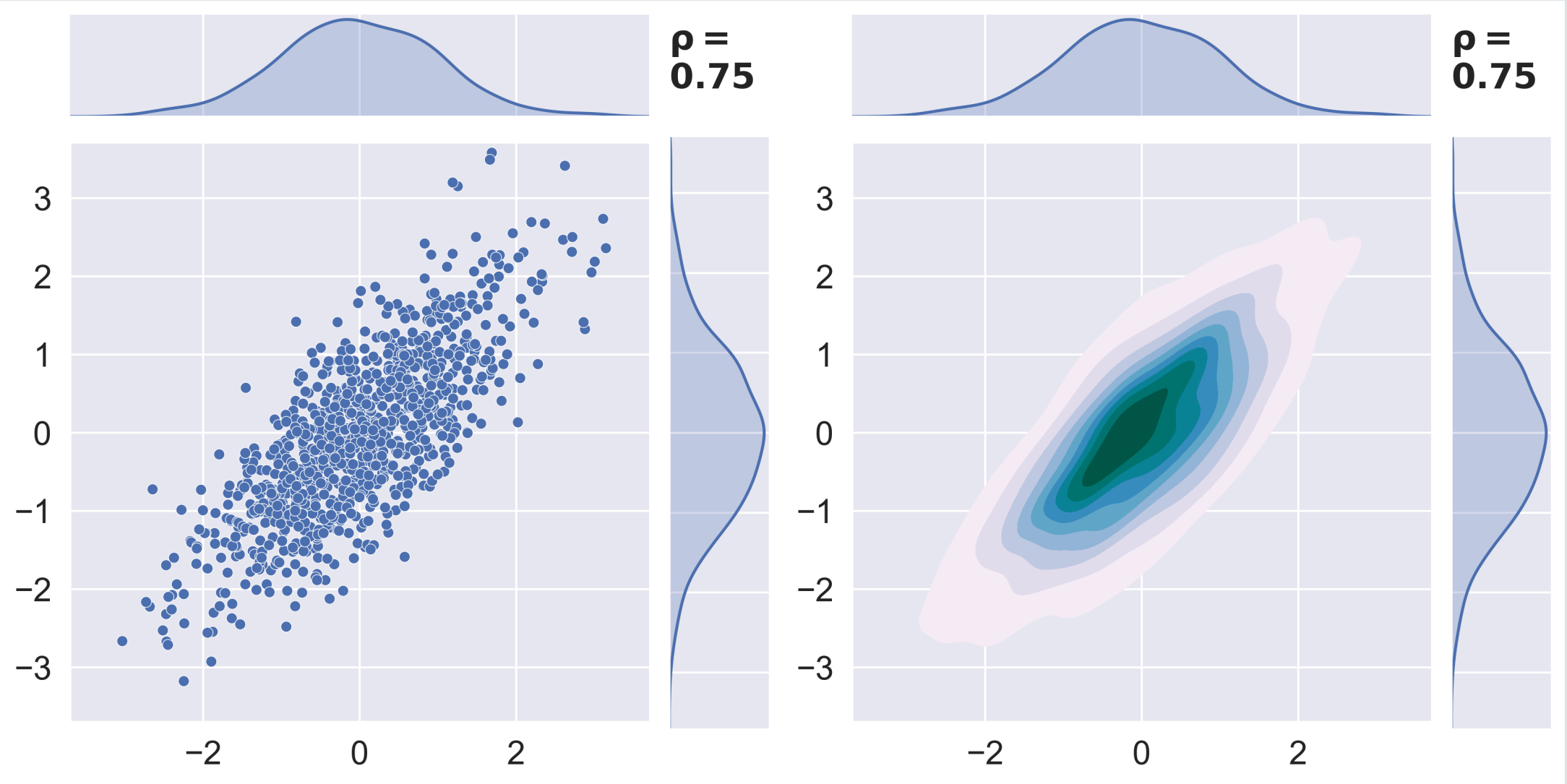

ρ = −0.5
ρ = −0.5
3
2
1
0
−1
−2
−3
−2
0
2
3
2
1
0
−1
−2
−3
−2
0
2

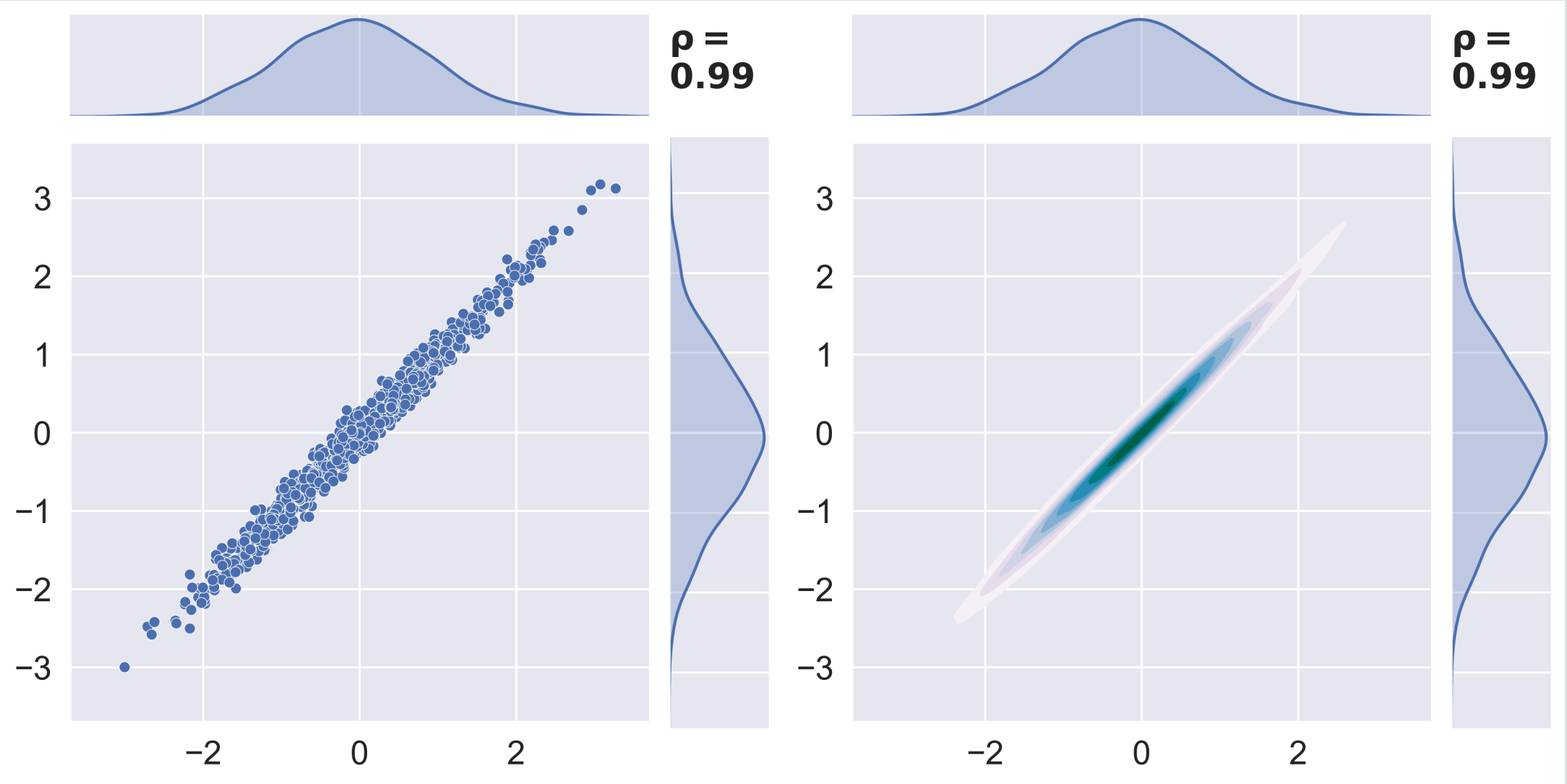

ρ =
−0.25
ρ =
−0.25
3
2
1
0
−1
−2
−3
−2
0
2
3
2
1
0
−1
−2
−3
−2
0
2


ρ = 0.0
ρ = 0.0
3
2
1
0
−1
−2
−3
−2
0
2
3
2
1
0
−1
−2
−3
−2
0
2


ρ = 0.25
ρ = 0.25
3
2
1
0
−1
−2
−3
−2
0
2
3
2
1
0
−1
−2
−3
−2
0
2


ρ = 0.5
ρ = 0.5
3
2
1
0
−1
−2
−3
−2
0
2
3
2
1
0
−1
−2
−3
−2
0
2


ρ = 0.75
ρ = 0.75
3
2
1
0
−1
−2
−3
−2
0
2
3
2
1
0
−1
−2
−3
−2
0
2


ρ = 0.99
ρ = 0.99
3
2
1
0
−1
−2
−3
−2
0
2
3
2
1
0
−1
−2
−3
−2
0
2

https://seaborn.pydata.org/generated/seaborn.scatterplot.html

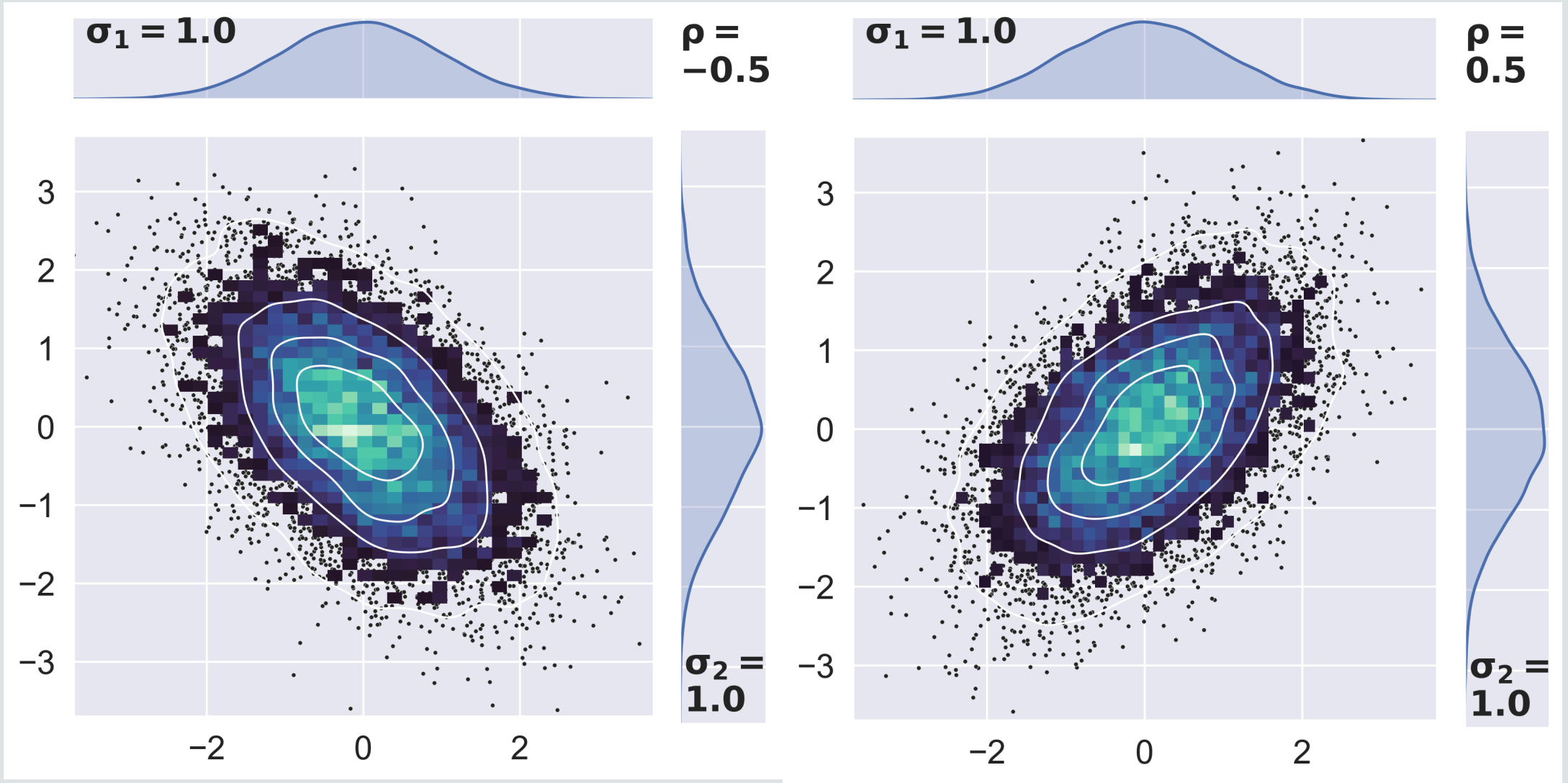
```
mu = np.array([0,0]) # mean vector
sigma1, sigma2, rho = 1.0,1.0,0.00 # variances and correlation
S = np.array([[sigma1*sigma1    ,rho*sigma1*sigma2] ,
              [rho*sigma1*sigma2,   sigma2*sigma2]])
X = multivariate_normal.rvs(mean=mu,cov=S,size=1000)

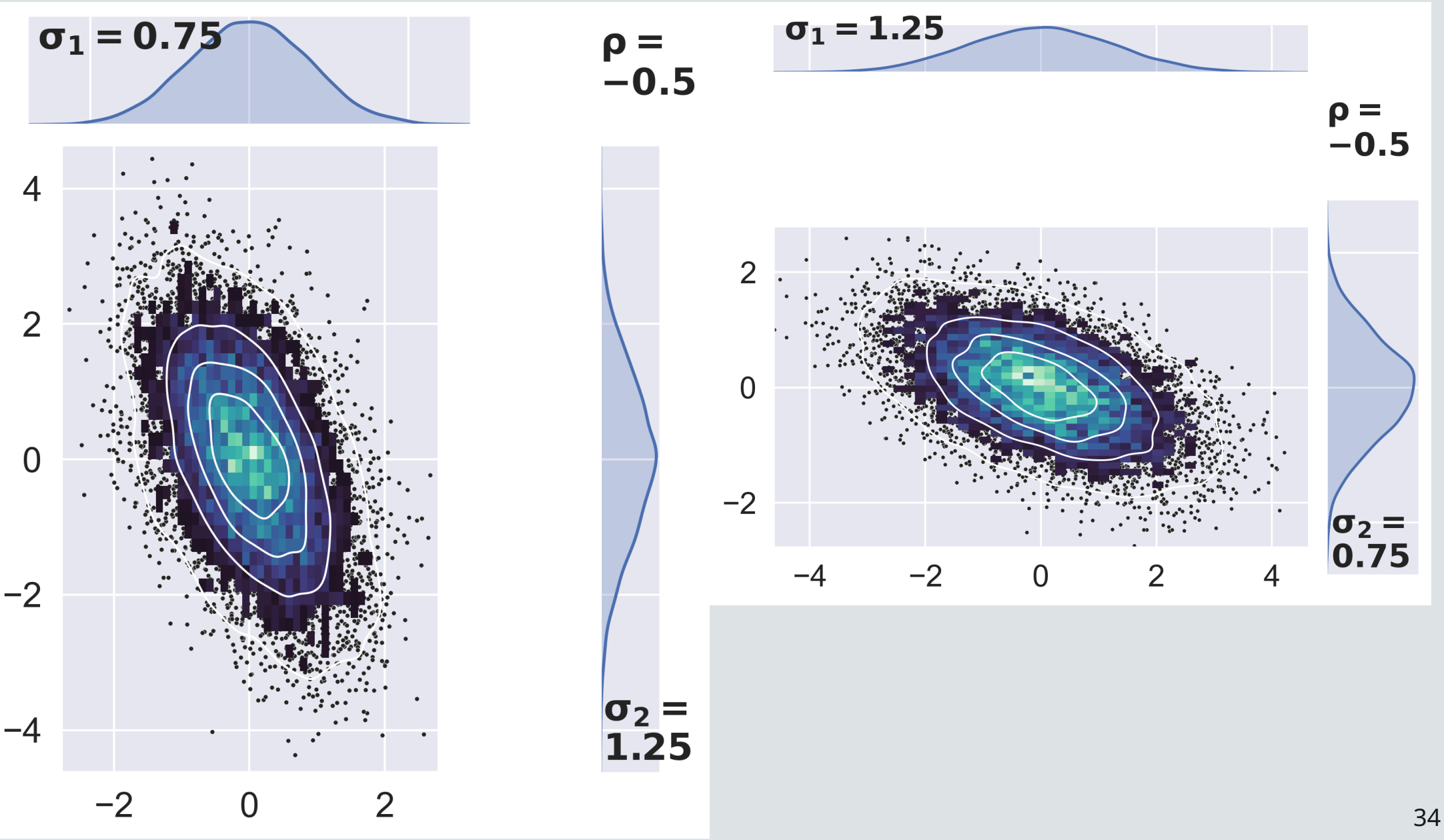
g = sns.JointGrid(x=X[:,0], y=X[:,1]) # grid of axes
sns.scatterplot(x=X[:,0],y=X[:,1],s=125,marker=".",linewidth=0.35,ax=g.ax_joir
sns.distplot(X[:,0], kde=True, hist=False, ax=g.ax_marg_x)
sns.distplot(X[:,1], kde=True, hist=False, ax=g.ax_marg_y, vertical=True)
kde_x = g.ax_marg_x.lines[0].get_xydata()
kde_y = g.ax_marg_y.lines[0].get_xydata()
g.ax_marg_x.fill_between(kde_x[:,0],kde_x[:,1],alpha=0.25)
g.ax_marg_y.fill_between(kde_y[:,0],kde_y[:,1],alpha=0.25)
g.ax_joint.set_xlim([-3.7,3.7]); g.ax_joint.set_ylim([-3.7,3.7])
g.ax_joint.set_aspect('equal')
plt.title(r'$\mathbf{\rho=}$'+'\n$\mathbf{'+str(rho)+'}$'+'\n', loc='left')
plt.show()
```

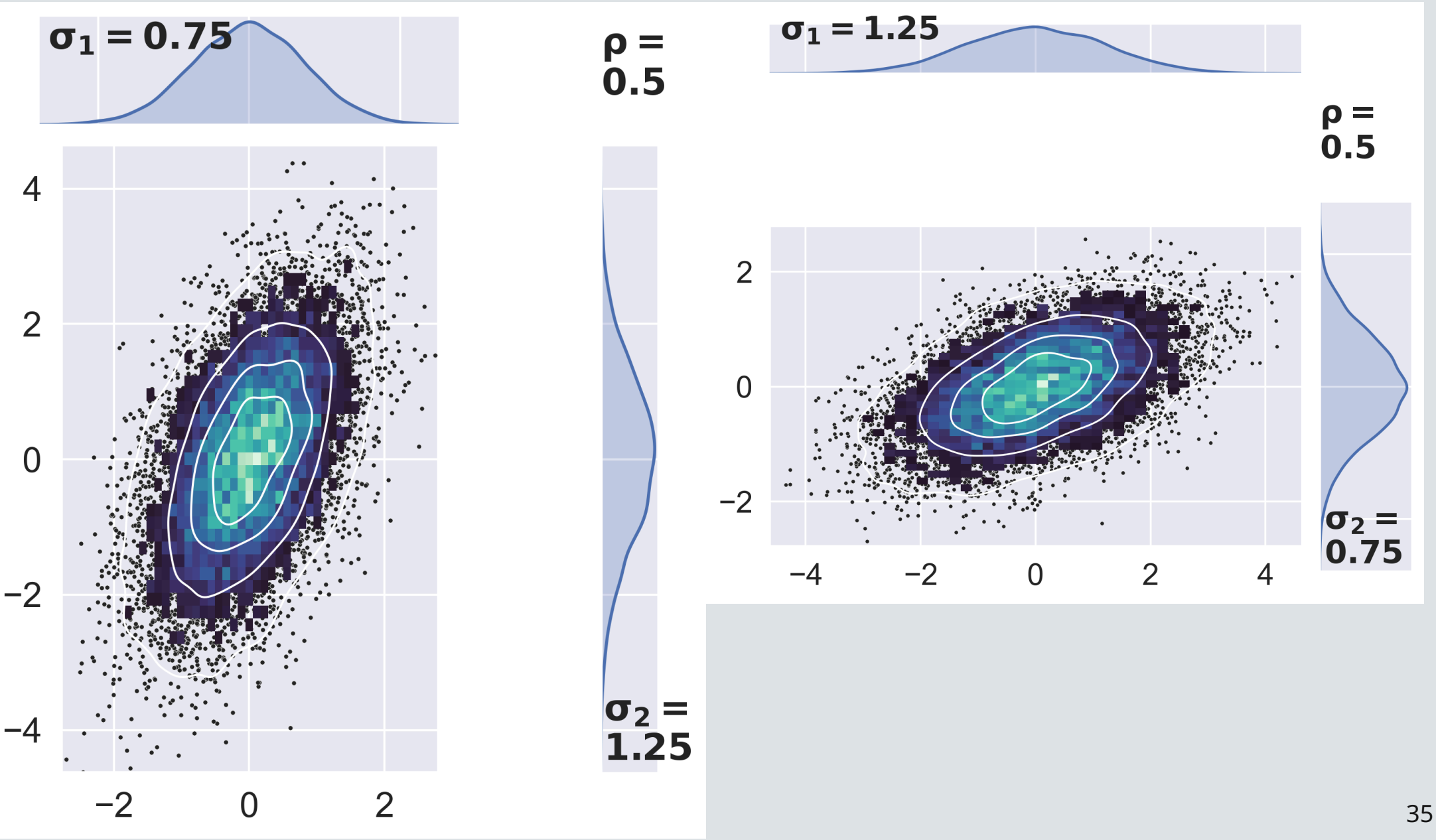
https://seaborn.pydata.org/generated/seaborn.kdeplot.html

```
mu = np.array([0,0]) # mean vector
sigma1, sigma2, rho = 1.0,1.0,0.00 # variances and correlation
S = np.array([[sigma1*sigma1    ,rho*sigma1*sigma2] ,
              [rho*sigma1*sigma2,   sigma2*sigma2]])
X = multivariate_normal.rvs(mean=mu,cov=S,size=1000)

g = sns.JointGrid(x=X[:,0], y=X[:,1]) # grid of axes
sns.kdeplot(X[:,0],X[:,1],cmap="PuBuGn",shade=True,shade_lowest=False,ax=g.ax_
sns.distplot(X[:,0], kde=True, hist=False, ax=g.ax_marg_x)
sns.distplot(X[:,1], kde=True, hist=False, ax=g.ax_marg_y, vertical=True)
kde_x = g.ax_marg_x.lines[0].get_xydata()
kde_y = g.ax_marg_y.lines[0].get_xydata()
g.ax_marg_x.fill_between(kde_x[:,0],kde_x[:,1],alpha=0.25)
g.ax_marg_y.fill_between(kde_y[:,0],kde_y[:,1],alpha=0.25)
g.ax_joint.set_xlim([-3.7,3.7]); g.ax_joint.set_ylim([-3.7,3.7])
g.ax_joint.set_aspect('equal')
plt.title(r'$\mathbf{\rho=}$'+'\n$\mathbf{'+str(rho)+'}$'+'\n', loc='left')
plt.show()
```


$\sigma_1 = 1.0$
$\rho = -0.5$
$\sigma_2 = 1.0$
$\sigma_1 = 1.0$
$\rho = 0.5$
$\sigma_2 = 1.0$
3
2
1
0
−1
−2
−3
−2
0
2


$\sigma_1 = 0.75$
$\rho = -0.5$
$\sigma_1 = 1.25$
$\rho = -0.5$
$\sigma_2 = 1.25$
$\sigma_2 = 0.75$
4
2
0
−2
−4
−2
0
2
−4
−2
0
2
4


$\sigma_1 = 0.75$
$\rho = 0.5$
$\sigma_1 = 1.25$
$\rho = 0.5$
$\sigma_2 = 1.25$
$\sigma_2 = 0.75$
4
2
0
−2
−4
−2
0
2
−4
−2
0
2
4

https://seaborn.pydata.org/generated/seaborn.histplot.html

```
g = sns.JointGrid(x=X[:,0], y=X[:,1])
g.ax_joint.set_aspect('equal')
g.ax_marg_x.set_aspect(np.sqrt(2*np.pi))
g.ax_marg_y.set_aspect(1/np.sqrt(2*np.pi))
sns.scatterplot(x=X[:,0], y=X[:,1], s=5, color=".15", ax=g.ax_joint)
sns.histplot(x=X[:,0], y=X[:,1], bins=50, pthresh=.1, cmap="mako", ax=g.ax_joint)
sns.kdeplot(x=X[:,0], y=X[:,1], levels=5, color="w", linewidths=1, ax=g.ax_joint)
sns.distplot(X[:,0], kde=True, hist=False, ax=g.ax_marg_x)
sns.distplot(X[:,1], kde=True, hist=False, ax=g.ax_marg_y, vertical=True)
kde_x = g.ax_marg_x.lines[0].get_xydata()
kde_y = g.ax_marg_y.lines[0].get_xydata()
g.ax_marg_x.fill_between(kde_x[:,0],kde_x[:,1],alpha=0.25)
g.ax_marg_y.fill_between(kde_y[:,0],kde_y[:,1],alpha=0.25)
g.ax_joint.set_xlim([-3.7*sigma1,3.7*sigma1])
g.ax_joint.set_ylim([-3.7*sigma2,3.7*sigma2])
plt.title(r'$\mathbf{\rho=}$'+'\n$\mathbf{'+str(rho)+'}$'+'\n', loc='left')
g.ax_marg_x.text(0.02, 0.7, r'$\mathbf{\sigma_1='+str(sigma1)+'}$', transform=g.a
g.ax_marg_y.text(0.05, 0.02, r'$\mathbf{\sigma_2=}$'+'\n$\mathbf{'+str(sigma2)+'}
plt.show()
```

Comments on plotting bivariate data

▷ A **scatter plot** (**scatterplot**) is a good starting point to visualize bivariate data.

▷ If the dataset is very large, a 2d **histogram** (**histplot**) might be preferable.

▷ A **contour plot** makes it possible to visualize the level sets of the bivariate density. It first requires an estimation of the density from the dataset. The function **kdeplot** achieves this by **kernel density estimation**.

▷ The last few plots combine a scatterplot, a histogram and a contour plot on the same figure, along with the marginal distributions (also estimated by kernel density estimation).

Bivariate Gaussian distribution

▷ The level sets of the bivariate Gaussian distribution are ellipses. In the independent case $\rho = 0$, they become circles.

▷ As the absolute value of ρ increases, these ellipses are squeezed towards the line defined by

$$x_2 = \operatorname{sign}(\rho)\frac{\sigma_2}{\sigma_1}(x_1 - \mu_1) + \mu_2$$

Replacing $\operatorname{sign}(\rho)$ by ρ in the above formula gives the **conditional expectation** of X_2 conditioned on $X_1 = x_1$, which corresponds to the *best linear unbiased prediction* of X_2 given a value of X_1.

Ill-conditioned covariance matrix

When estimating a high-dimensional covariance matrix on financial data, it is very likely that the matrix will be **ill-conditioned** due to virtual multicollinearity.

Recall from Chapter 2 that the **condition number** of a matrix is given, in the case of a positive-definite, symmetric square matrix such as a covariance matrix $\mathbf{\Sigma}$ by the ratio $\lambda_1(\mathbf{\Sigma})/\lambda_D(\mathbf{\Sigma})$ of the largest eigenvalue over the smallest eigenvalue.

Numerical algorithms such as the Cholesky decomposition might struggle to return reliable results if the covariance matrix is very ill-conditioned.

Covariance matrix regularization

A common approach is to **regularize** the matrix, which means replacing it by a very close yet much better conditioned matrix. The simplest regularization technique is to replace $\mathbf{\Sigma}$ by

$$\mathbf{\Sigma}_\varepsilon := \frac{1}{1+\varepsilon}\left(\mathbf{\Sigma} + \varepsilon \mathbf{I}\right)$$

This leaves the eigenvectors of $\mathbf{\Sigma}$ unchanged but replaces the eigenvalues $\lambda_d(\mathbf{\Sigma}), d = 1, \ldots, D$ of $\mathbf{\Sigma}$ by

$$\lambda_d(\mathbf{\Sigma}_\varepsilon) = \frac{\lambda_d(\mathbf{\Sigma}) + \varepsilon}{1+\varepsilon}$$

Covariance matrix regularization

The new condition number is

$$\mathrm{cond}(\boldsymbol{\Sigma}_\varepsilon) = \frac{\lambda_1(\boldsymbol{\Sigma}) + \varepsilon}{\lambda_D(\boldsymbol{\Sigma}) + \varepsilon} \ll \frac{\lambda_1(\boldsymbol{\Sigma})}{\lambda_D(\boldsymbol{\Sigma})} = \mathrm{cond}(\boldsymbol{\Sigma})$$

Indeed, one can show that

$$\mathrm{cond}(\boldsymbol{\Sigma}_\varepsilon) = 1 + \frac{\lambda_D(\boldsymbol{\Sigma})}{\lambda_D(\boldsymbol{\Sigma}) + \varepsilon}(\mathrm{cond}(\boldsymbol{\Sigma}) - 1)$$

where $\mathrm{cond}(\boldsymbol{\Sigma}) \geq 1$ and $\frac{\lambda_D(\boldsymbol{\Sigma})}{\lambda_D(\boldsymbol{\Sigma})+\varepsilon} < 1$

This is an example of **shrinkage estimator**.

5.2 Simulation of correlated non-Gaussian variables

The Iman-Conover method

Unlike the Gaussian case, there is **no simple way** to simulate a multivariate random vector with prescribed covariance matrix in the non-Gaussian case. We are going to describe the most common method, known as the **Iman-Conover method**, to attempt to solve this problem.

Let $[X_1, \ldots, X_D]^\top$ be a vector of (non-Gaussian) random variables. We want to simulate it while enforcing a prescribed covariance matrix Σ between them.

1) First, simulate a vector $[N_1, \ldots, N_D]^\top$ of standard **Gaussian** variables with covariance matrix $\tilde{\Sigma}$ (which will be defined later) using the algorithm introduced previously.

The Iman-Conover method

2) Let Φ be the cumulative distribution function (CDF) of the standard Gaussian variable. We know that for each $d = 1, \ldots, D$

$$U_d := \Phi(N_d)$$

is a uniformly-distributed random variable.

3) For each $d = 1, \ldots, D$, let F_{X_d} be the cumulative distribution function (CDF) of X_d. Suppose that its inverse $F_{X_d}^{-1}$ exists. We know from the **inversion method** that for each $d = 1, \ldots, D$

$$F_{X_d}^{-1}(U_d)$$

has the same distribution as X_d.

The Iman-Conover method

Since the original (Gaussian) vector $[N_1, \ldots, N_D]^\top$ is correlated (with correlation matrix $\tilde{\Sigma}$), the transformed vector

$$\left[F_{X_1}^{-1}\left(\Phi(N_1)\right), \ldots, F_{X_D}^{-1}\left(\Phi(N_D)\right)\right]^\top$$

is also **correlated**. To sum up, this method simulates a correlated random vector with marginals distributed as $X_1, \ldots, X_D$.

⚠ **Problem:** since the compound transforms $F_{X_d}^{-1} \circ \Phi$ are (generally) nonlinear, the correlation matrix Σ of the resulting vector $[X_1, \ldots, X_D]^\top$ is **different from** the correlation matrix $\tilde{\Sigma}$ of the Gaussian vector $[N_1, \ldots, N_D]^\top$. As a result, an **optimization** is needed to find the matrix $\tilde{\Sigma}$ such that the output vector has the prescribed covariance matrix Σ.

The Iman-Conover method

⚠ **Problem:** In general, not all covariance matrices Σ can be attained by optimizing the Gaussian covariance matrix $\tilde{\Sigma}$.

This method is known as the **Iman-Conover method** in actuarial science, and is also known as the **NORTA method** (**NORmal To Anything**).

[1] R. Iman and W. Conover. "A distribution-free approach to inducing rank correlation among input variables". In: *Communications in Statistics-Simulation and Computation* 11.3 (1982), pp. 311-334.

[2] M. Cario and B. Nelson. *Modeling and generating random vectors with arbitrary marginal distributions and correlation matrix*. Tech. rep. Northwestern University, Illinois, 1997.

Example: correlated uniforms

As an example, let's apply this method to simulate two correlated uniform random variables.

Let U_1 and U_2 be two uniform random variables with correlation ρ.
In order to simulate U_1 and U_2, we first simulate two standard Gaussian random variables G_1 and G_2 with correlation $\tilde{\rho}$. We know that they can be obtained by

$$\begin{aligned} G_1 &:= N_1 \\ G_2 &:= \tilde{\rho} N_1 + \sqrt{1 - \tilde{\rho}^2} N_2 \end{aligned}$$

where N_1 and N_2 are two independent standard Gaussian variables.

Correlated uniform random variables

Recall that Φ is the CDF of a standard Gaussian variable. We can define

$$U_1 := \Phi(G_1) \qquad U_2 := \Phi(G_2)$$

We know that U_1 and U_2 defined in this way are uniformly distributed. According to the Iman-Conover method, we should furthermore apply the inverse CDF F_U^{-1} of a uniform random variable to obtain U_1 and U_2, but since $F_U(x) = F_U^{-1}(x) = x$ for $0 \leq x \leq 1$, we can ignore this step on this particular example.

Let ρ be the correlation between U_1 and U_2. It is different from the correlation $\tilde{\rho}$ between G_1 and G_2, and the final task is to find how to define $\tilde{\rho}$ (as a function of ρ) to obtain the desired correlation between U_1 and U_2.

Spearman's rank correlation

Let X and Y be two random variables with CDF F_X and F_Y respectively. We know that the two variables $U_1 := F_X(X)$ and $U_2 := F_Y(Y)$, also known as **rank variables**, are uniformly distributed. By definition, **Spearman's rank correlation** ρ_S between X and Y is the (Pearson) correlation between $F_X(X)$ and $F_Y(Y)$:

$$\rho_S := \frac{\mathbb{C}\mathrm{ov}(U_1, U_2)}{\sigma(U_1)\sigma(U_2)}$$

where $\mathbb{C}\mathrm{ov}(U_1, U_2) = \mathbb{E}[U_1 U_2] - \mathbb{E}[U_1]\mathbb{E}[U_2]$ is the covariance between U_1 and U_2, $\sigma(U_1)$ is the standard deviation of U_1 (equal to 1/12), and $\sigma(U_2)$ is the standard deviation of U_2 (also equal to 1/12).

Spearman correlation vs. Pearson correlation

In general, Spearman's rank correlation and the classical (Pearson) correlation are different. Going back to the application of the Iman-Conover method to two uniform random variables, the problem can be described as finding the correlation $\tilde{\rho}$ between the two Gaussian variables G_1 and G_2 such that the **Spearman correlation** between G_1 and G_2 is equal to ρ. Fortunately, in this Gaussian case there is a formula which relates the Pearson correlation and the Spearman correlation:

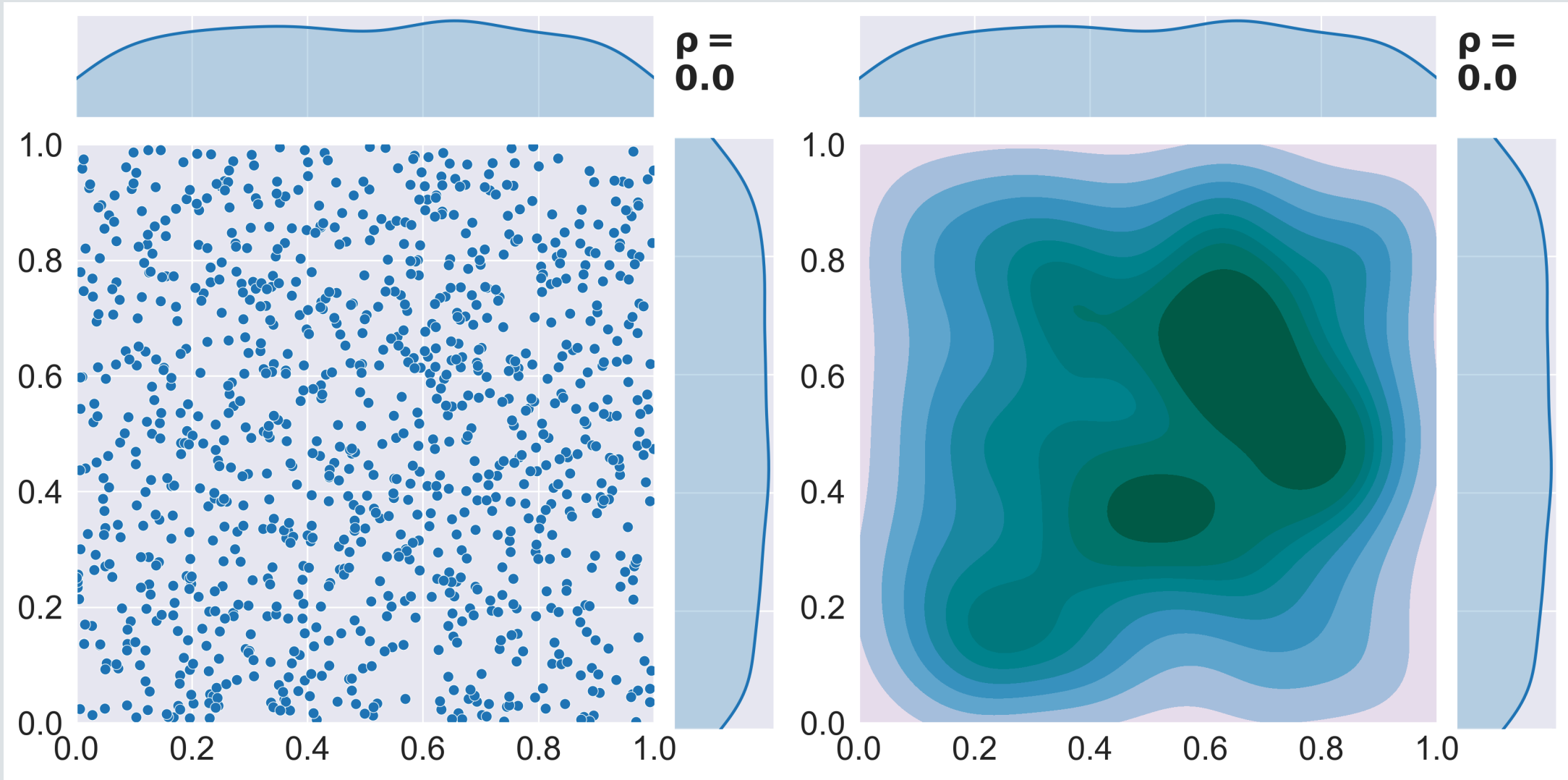
$$\rho = \frac{6}{\pi} \arcsin\left(\frac{\tilde{\rho}}{2}\right)$$

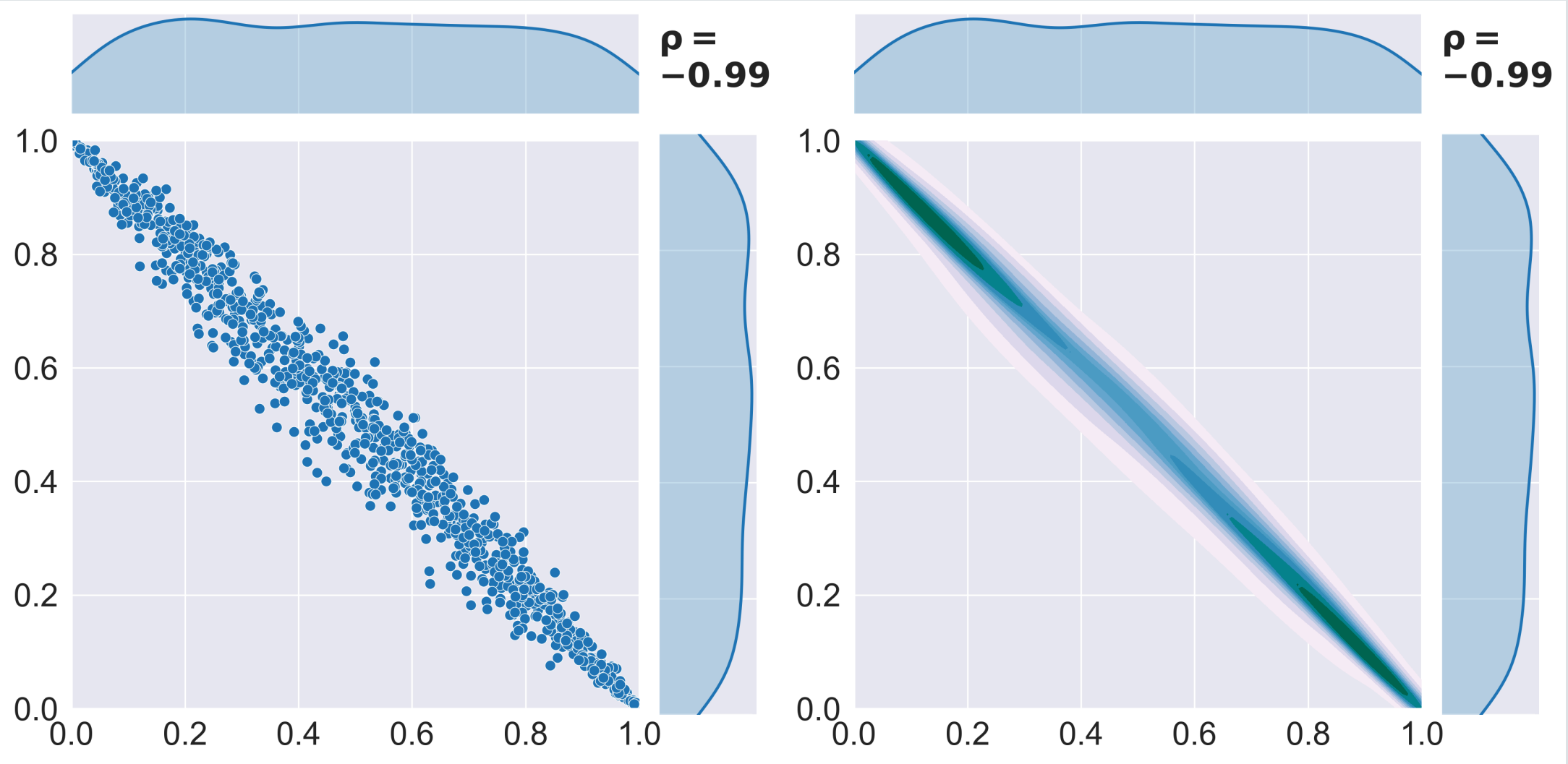
Simulate correlated uniform random variables

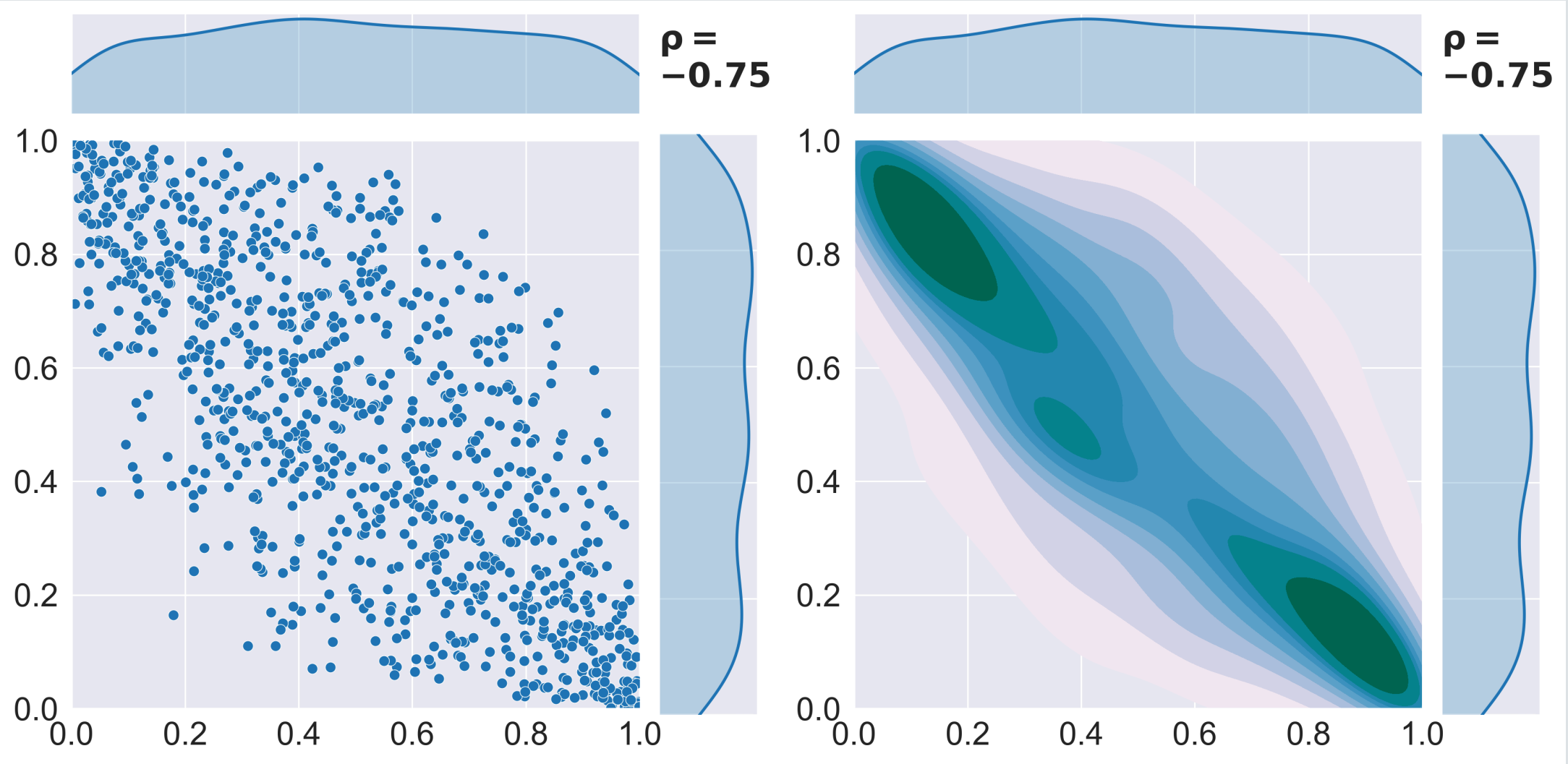
We can now summarise the algorithm to simulate two correlated uniform random variables with prescribed correlation ρ:

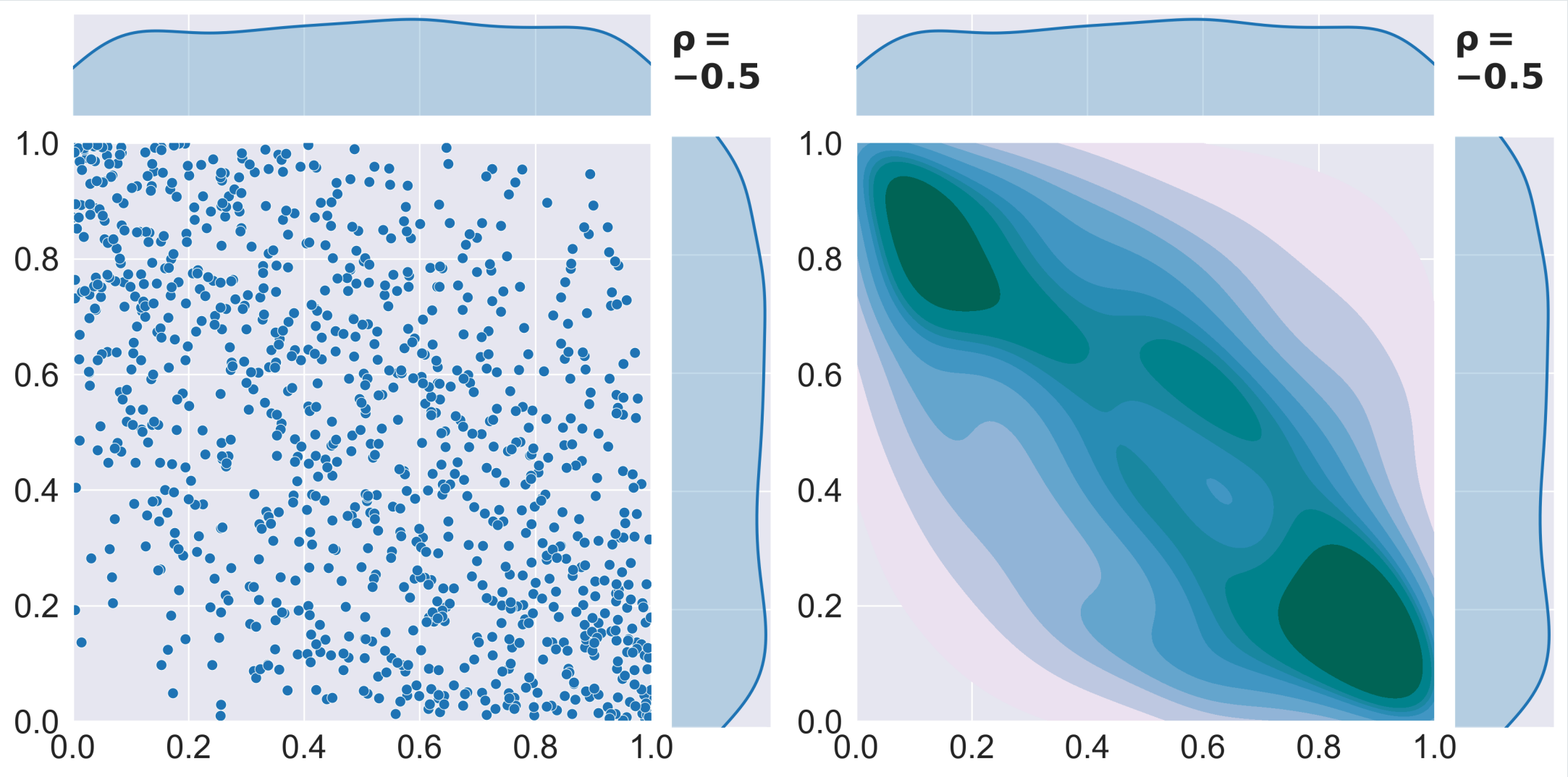
▷ Define $\tilde{\rho} = 2\sin(\rho\pi/6)$
▷ Simulate two Gaussian variables G_1 and G_2 with correlation $\tilde{\rho}$ using the previously described algorithm
▷ Define $U_1 := \Phi(G_1)$ and $U_2 := \Phi(G_2)$ where Φ is the standard Gaussian CDF.

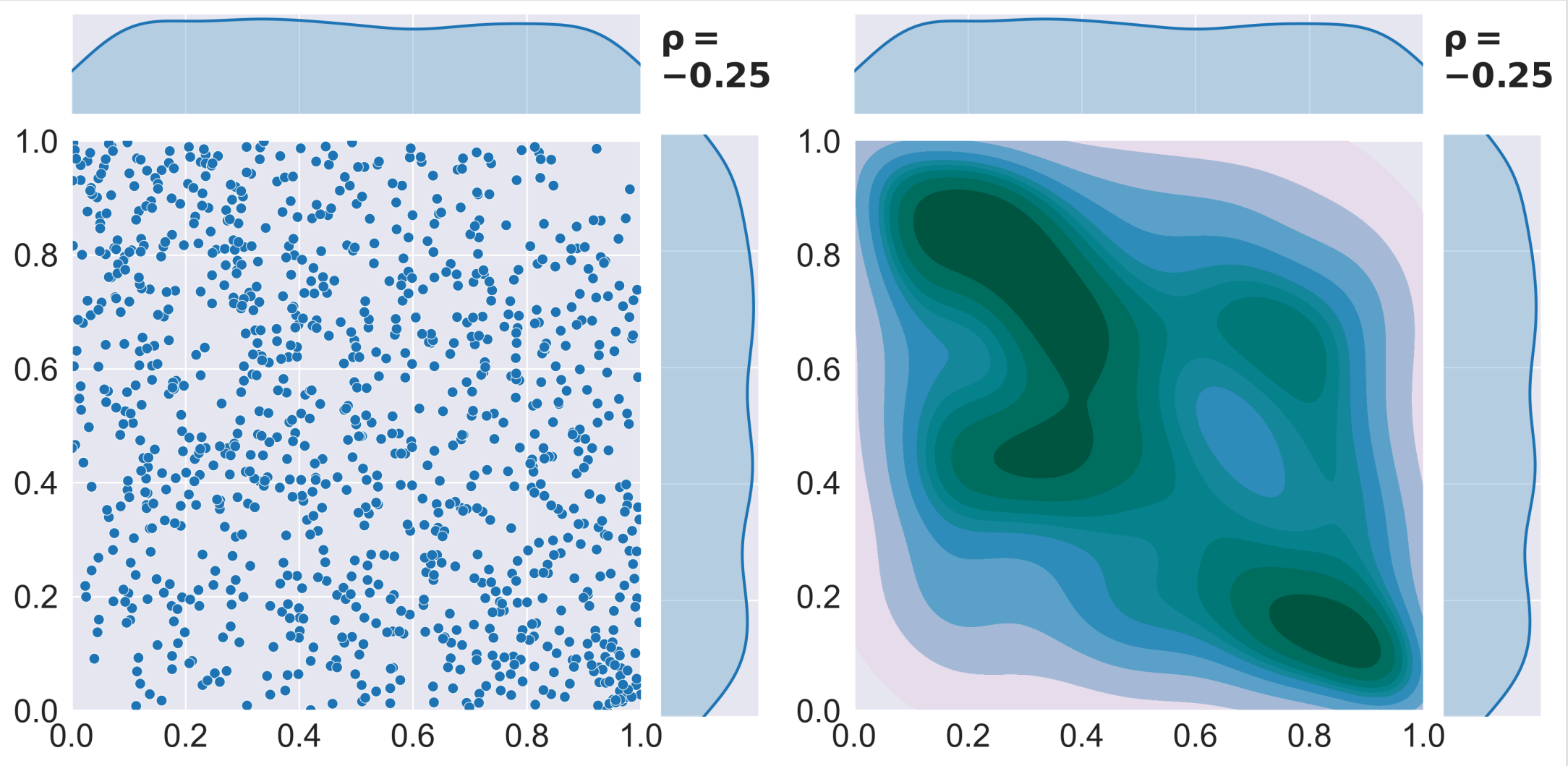
Then $\mathbf{U_1}$ **and** $\mathbf{U_2}$ **are uniform random variables with correlation** ρ.

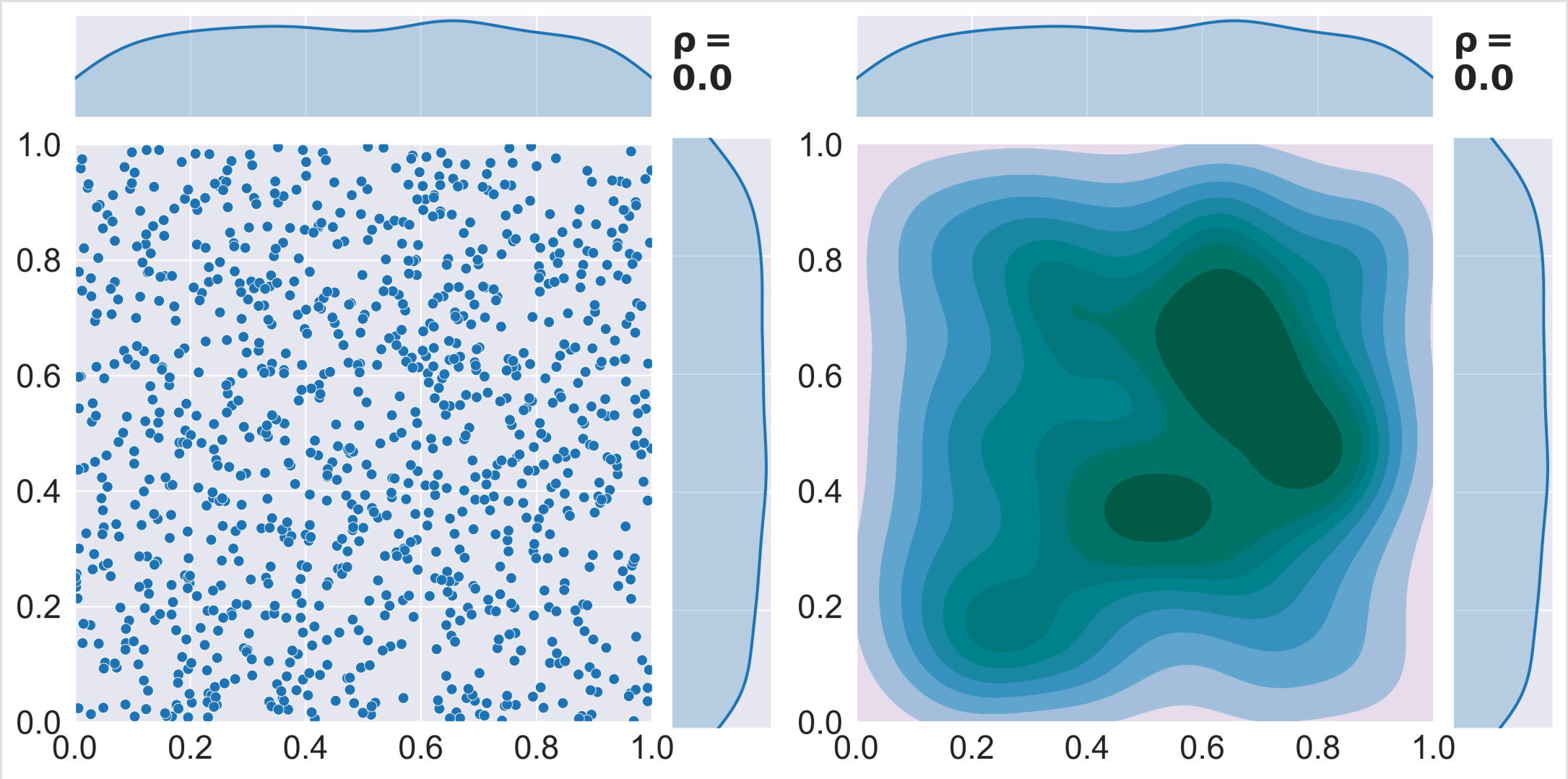

ρ =
0.0
ρ =
0.0
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

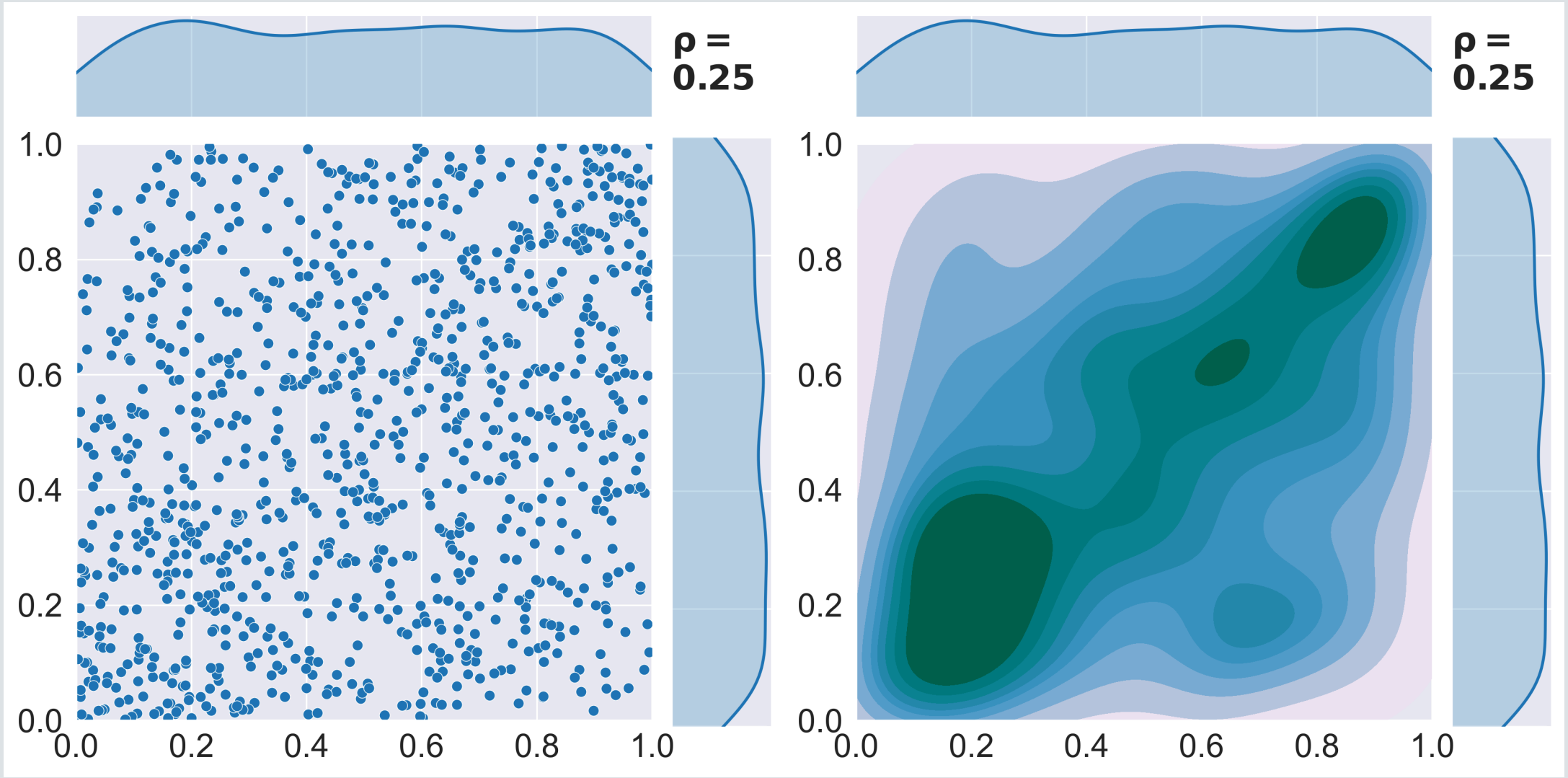

ρ =
−0.99
ρ =
−0.99
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

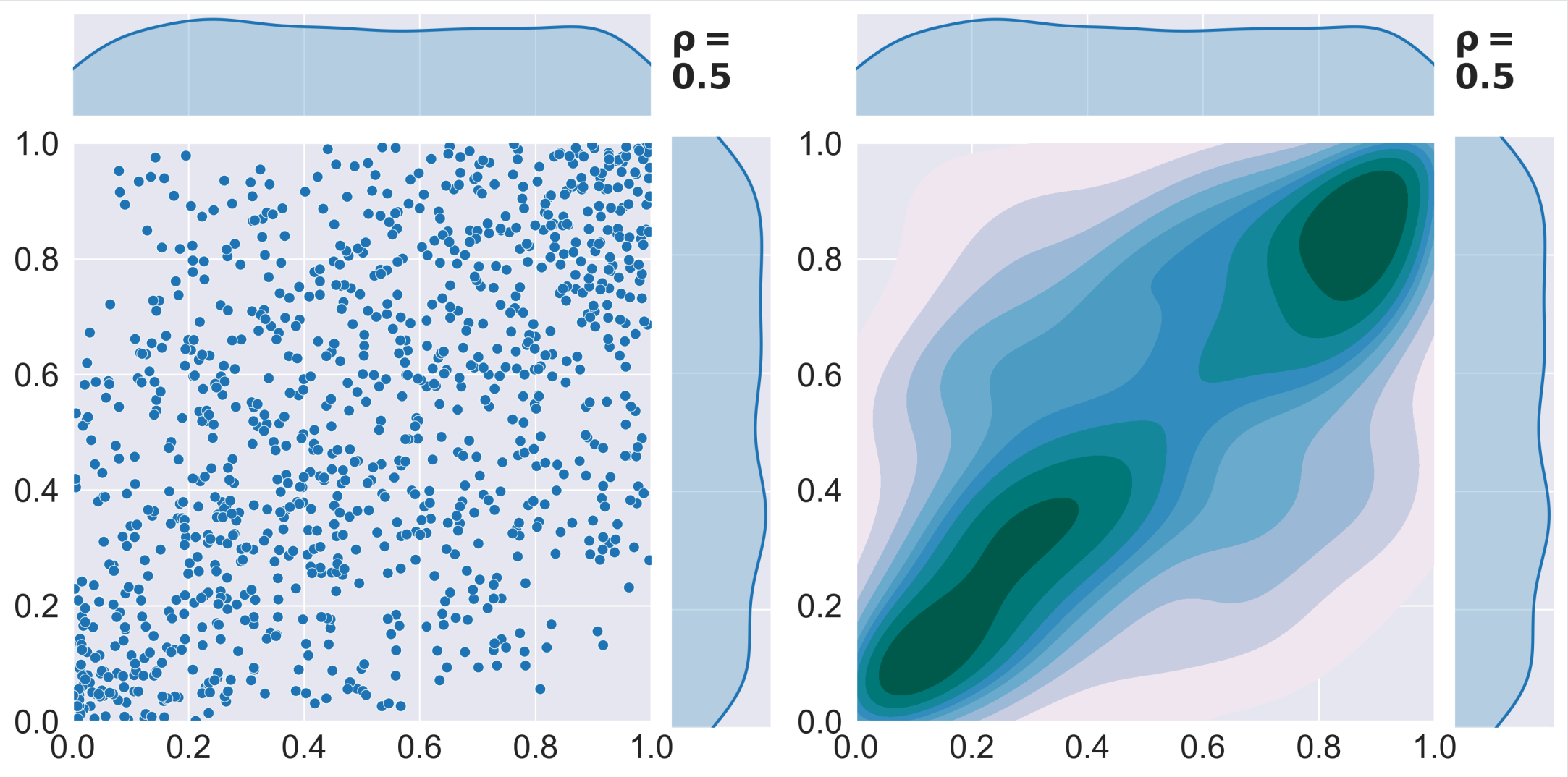

ρ =
−0.75
ρ =
−0.75
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

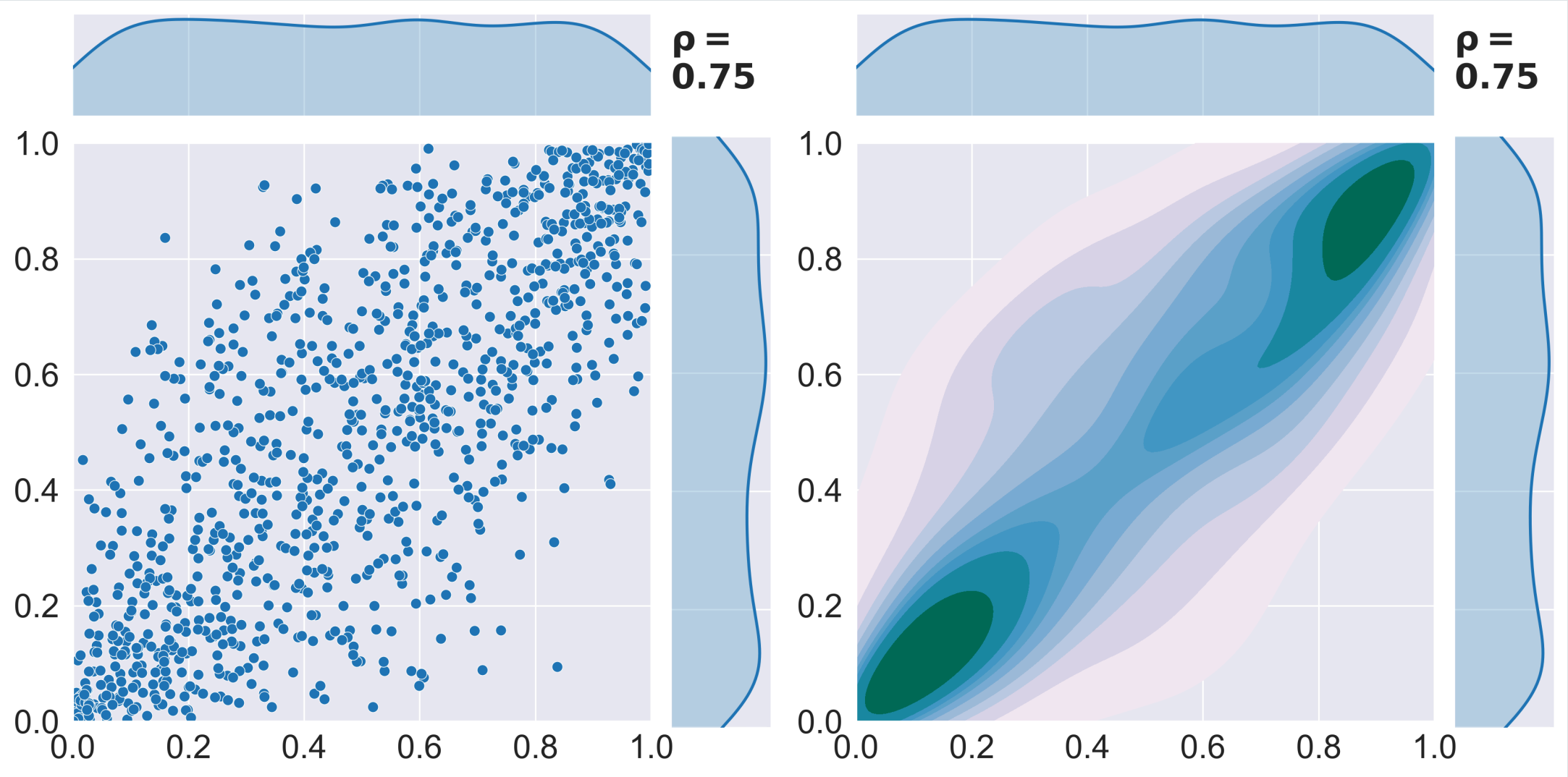

ρ =
−0.5
ρ =
−0.5
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

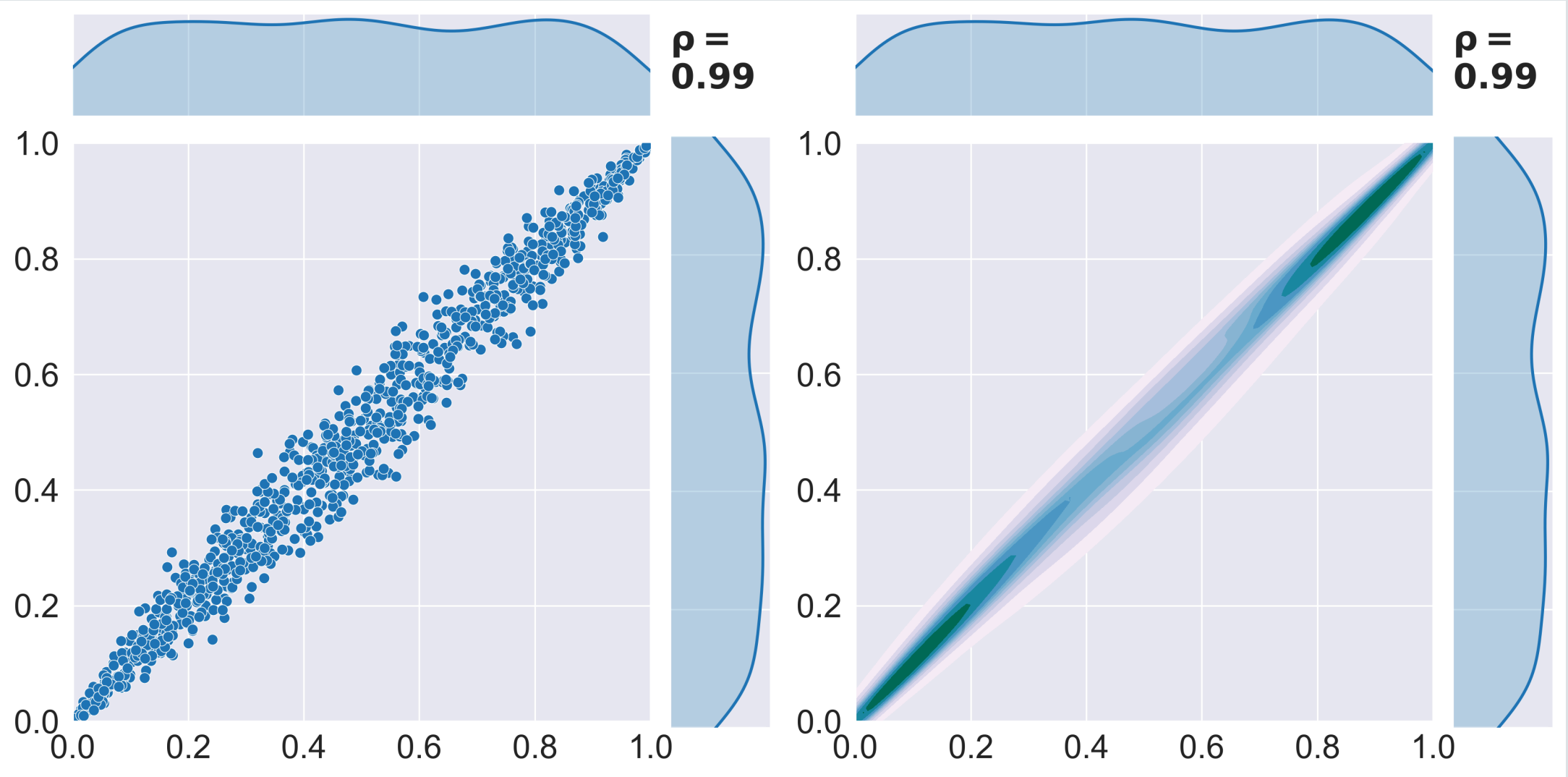

ρ =
−0.25
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
ρ =
−0.25
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


ρ =
0.0
ρ =
0.0
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


ρ =
0.25
ρ =
0.25
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


ρ =
0.5
ρ =
0.5
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


ρ =
0.75
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
ρ =
0.75
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


ρ =
0.99
ρ =
0.99
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

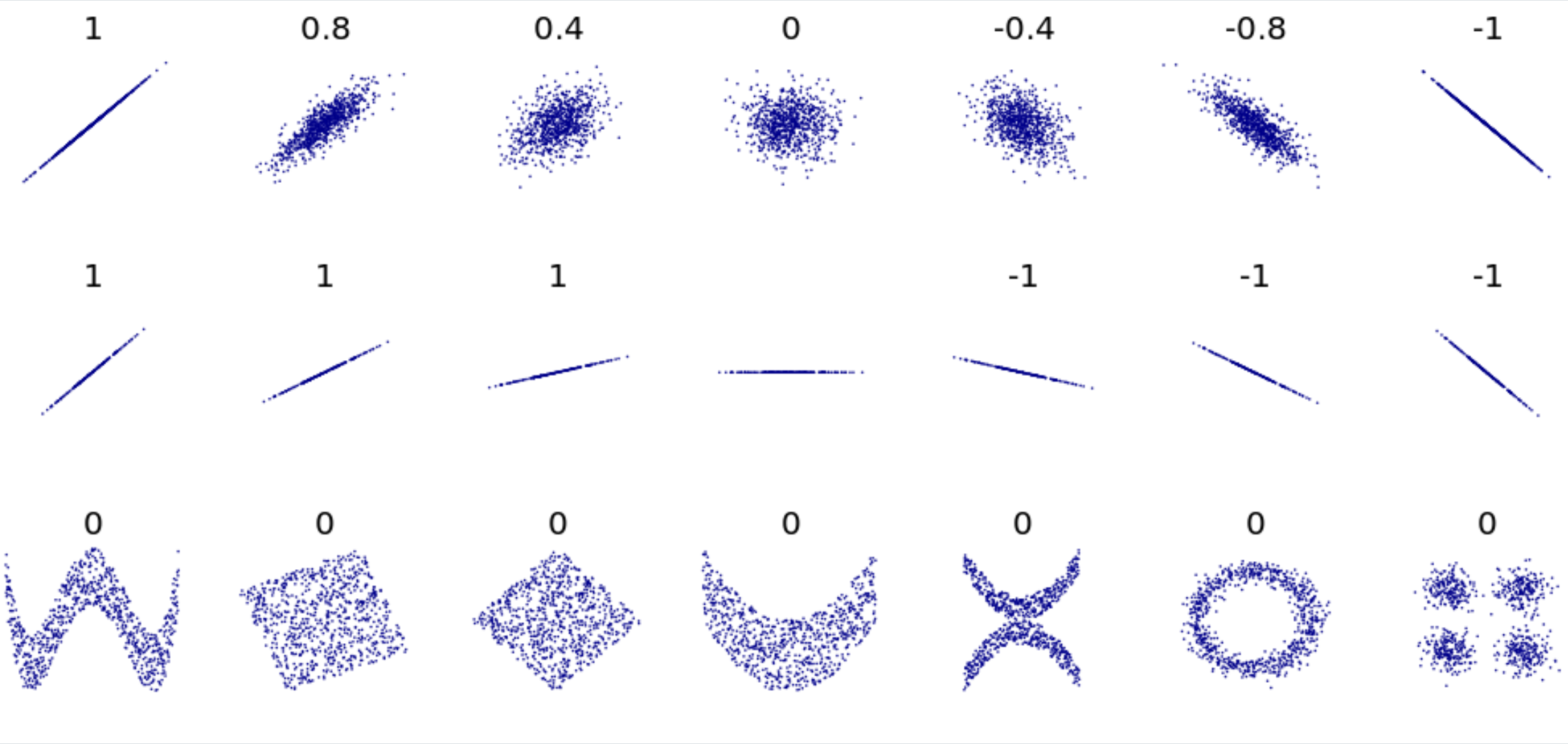
Limitations of correlations

In the Gaussian case, the **dependence structure** of a random vector is fully determined by its covariance matrix. In the non-Gaussian case, this is not the case anymore:

▷ uncorrelated variables are *not necessarily independent*,

▷ the same marginal distributions and covariances can be achieved by *infinitely many* different joint distributions.

As a result, the intuition we get about the meaning of correlation in the Gaussian context does not apply to non-Gaussian variables. In other words, **correlations are not a great way of defining or measuring the dependence structure between (non-Gaussian) variables**. More suitable tools are needed, such as **copulas**.


1
0.8
0.4
0
-0.4
-0.8
-1
1
1
1
-1
-1
-1
0
0
0
0
0
0
0

5.3 Copulas

Bivariate copula

Let X and Y be two random variables. Let $F_X(x) = \mathbb{P}(X \leq x)$ and $F_Y(y) = \mathbb{P}(Y \leq y)$ be the **marginal CDFs** of X and Y respectively, and let $F_{X,Y}(x, y) = \mathbb{P}(X \leq x, Y \leq y)$ be the **joint CDF** of (X, Y). According to **Sklar's theorem**, there exists a **copula** function $\mathcal{C} = \mathcal{C}(u, v)$ such that the joint CDF $F_{X,Y}$ can be decomposed as follows:

$$F_{X,Y}(x, y) = \mathcal{C}(F_X(x), F_Y(y))$$

By definition, a **copula** is a **multivariate cumulative distribution function for which the marginal probability distribution of each variable is uniform on the interval [0, 1]**

Multivariate copula

The same decomposition of joint CDFs holds in higher dimension. Let $X_1, X_2, \ldots, X_D$ be a D-dimensional random vector. For each $d = 1, 2, \ldots, D$, let F_{X_d} be the CDF of the random variable X_d. Finally, let

$$F(x_1, x_2, \ldots, x_D) = \mathbb{P}(X_1 \leq x_1, X_2 \leq x_2, \ldots, X_D \leq x_D)$$

be the joint CDF of the vector $X_1, X_2, \ldots, X_D$. According to **Sklar's theorem**, there exists a **copula** $\mathcal{C} = \mathcal{C}(u_1, u_2, \ldots, u_D)$ such that, the joint CDF F can be decomposed as

$$F(x_1, x_2, \ldots, x_D) = \mathcal{C}(F_{X_1}(x_1), F_{X_2}(x_2), \ldots, F_{X_D}(x_D))$$

Dependence modelling using copulas

Sklar's theorem tells us that a multivariate distribution can be decomposed into two parts:

▷ the modelling of the **marginal distributions**, determined by the univariate cumulative distribution functions F_{X_d}, $d = 1, 2, \ldots, D$

▷ the modelling of the **dependence structure**, determined by the copula $\mathcal{C}$

This separation between marginal distributions and the dependence structure is what makes copulas very suitable for dependence modelling. By contrast, alternative measures of dependence such as Pearson's correlation are affected by both the dependence structure AND the marginal distributions (which is the reason why the simulation of non-Gaussian random variables with prescribed correlation coefficient is hard and sometimes impossible).

Independence copula

For simplicity we focus on bivariate copulas. The simplest example of copula is the so-called **independence copula**, defined by

$$\mathcal{C}(u, v) = u \times v$$

This is the copula of random variables which are **independent** of each other. Indeed, if X and Y are two independent random variables with joint CDF $F_{X,Y}$ and marginal CDFs F_X and F_Y respectively, then

$$F_{X,Y}(x, y) = \mathcal{C}(F_X(x), F_Y(y)) = F_X(x) \times F_Y(y)$$

for all x, y.

Copula bounds

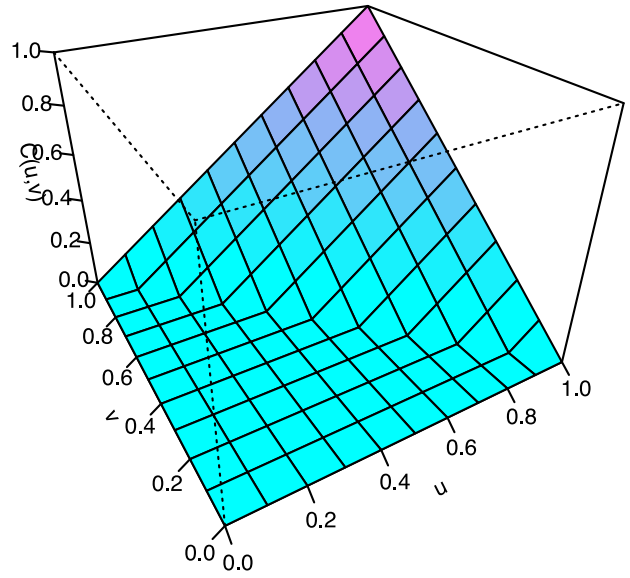
The **Fréchet-Hoeffding theorem** states that the following bounds hold for any copula $\mathcal{C}$:

$$\max(u+v-1,0) \leq \mathcal{C}(u,v) \leq \min(u,v)$$

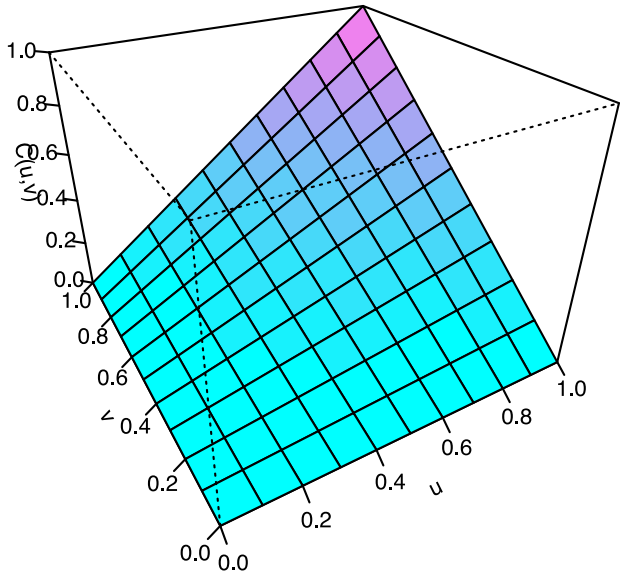
▷ The upper bound $M(u,v) := \min(u,v)$ is a copula which corresponds to **comonotonic random variables** (perfect positive dependence, meaning that $(X,Y) = (F_X^{-1}(\mathbf{U}), F_Y^{-1}(\mathbf{U}))$ using the *same* uniform random variable $\mathbf{U}$).

▷ The lower bound $W(u,v) := \max(u+v-1,0)$ is a copula which corresponds to **countermonotonic random variables** (perfect negative dependence, meaning that $(X,Y) = (F_X^{-1}(\mathbf{U}), F_Y^{-1}(\mathbf{1}-\mathbf{U}))$ using the *same* uniform random variable $\mathbf{U}$).

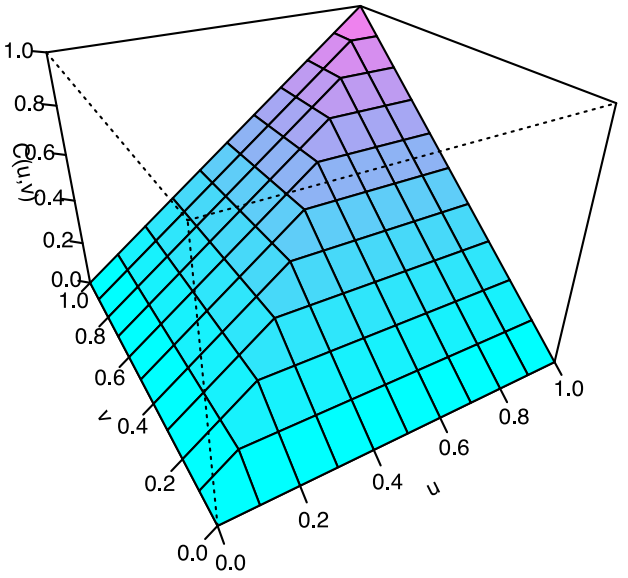
Minimum



Independence



Maximum



Archimedean copula

A copula $\mathcal{C}$ is called **Archimedean** if it admits the representation

$$\mathcal{C}(u, v) = \phi^{-1}(\phi(u) + \phi(v))$$

where the **generator** ϕ is a continuous, decreasing and convex function such that $\phi(1) = 0$.

The simplest example of Archimedean copula is the independence copula $\mathcal{C}(u, v) = u \times v$, obtained by choosing the generator $\phi(u) = -\log(u)$.

Clayton copula

Another example of Archimedean copula is the **Clayton copula**, obtained by choosing the generator $\phi_\theta(u) = \frac{1}{\theta}(u^{-\theta} - 1)$. It is given by

$$\mathcal{C}_\theta(u, v) = \max\left[\left(u^{-\theta} + v^{-\theta} - 1\right)^{-1/\theta}, 0\right]$$

where $\theta \in [-1, \infty) \backslash \{0\}$ is a parameter, which controls the strength of the **lower tail dependence**. This class of copulas contain the following three important particular cases:

$\theta = -\mathbf{1}$: the lower Fréchet bound $\mathcal{C}_\theta(u, v) = \max(u + v - 1, 0)$

$\theta \to \mathbf{0}$: the independence copula $\mathcal{C}_\theta(u, v) = uv$

$\theta \to \infty$: the upper Fréchet bound $\mathcal{C}_\theta(u, v) = \min(u, v)$

Simulation from a copula

A general method to simulate draws from a chosen copula $\mathcal{C}$ is known as **conditional sampling**. Let U and V be two standard uniform random variables. Consider the conditional distribution

$$c_u(v) = \mathbb{P}(V \leq v | U = u)$$

c_u is the CDF of V *conditionally on the event* $U = u$ where $u \in [0, 1]$. One can show that $c_u(v) = \frac{\partial C}{\partial u}(u, v)$. We can then simulate two uniform random variables with prescribed copula by the **inversion method**:

▷ Generate two independent uniform random variable simulations $(u_1, u_2) \in [0, 1]^2$

▷ Set $u = u_1$ and $v = c_{u_1}^{-1}(u_2)$

Simulation from the Clayton copula

As an example, we apply the conditional sampling technique to the Clayton copula with $\theta > 0$. We compute

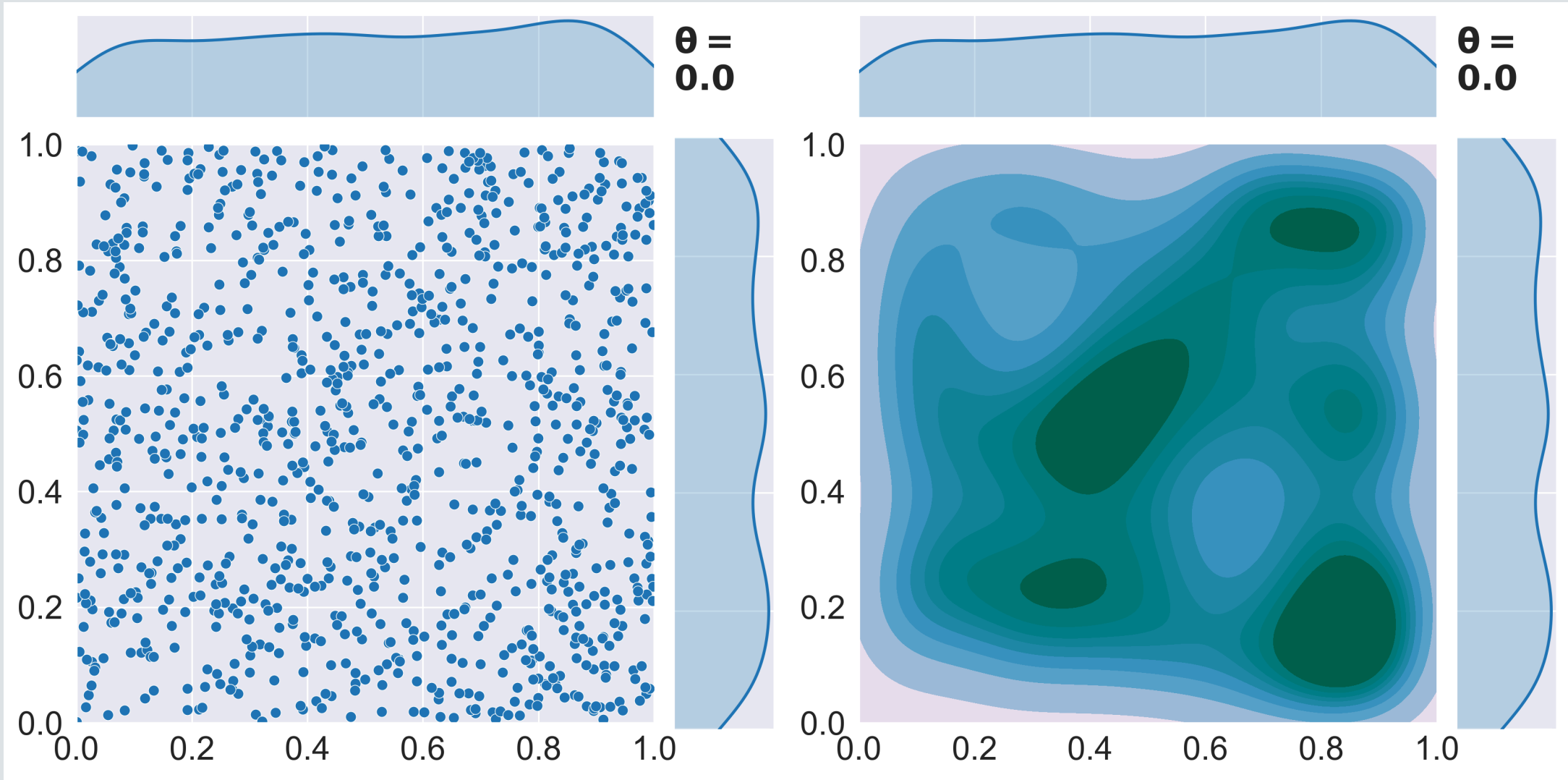
$$\mathcal{C}_\theta(u, v) = \left(u^{-\theta} + v^{-\theta} - 1\right)^{-1/\theta}$$

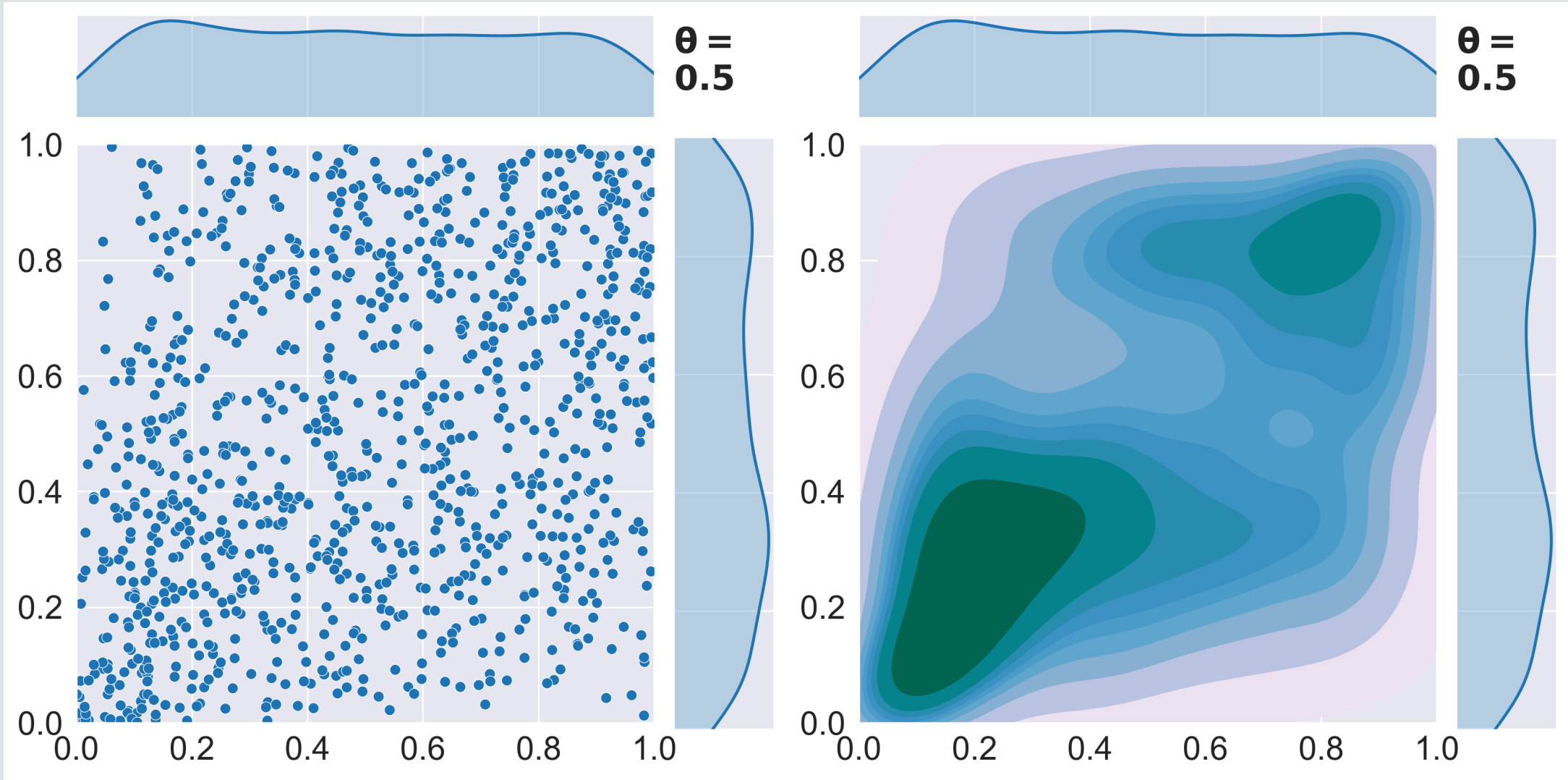
$$c_{\theta,u}(v) := \frac{\partial \mathcal{C}_\theta}{\partial u}(u, v) = u^{-\theta-1}\left(u^{-\theta} + v^{-\theta} - 1\right)^{-\frac{1}{\theta}-1}$$

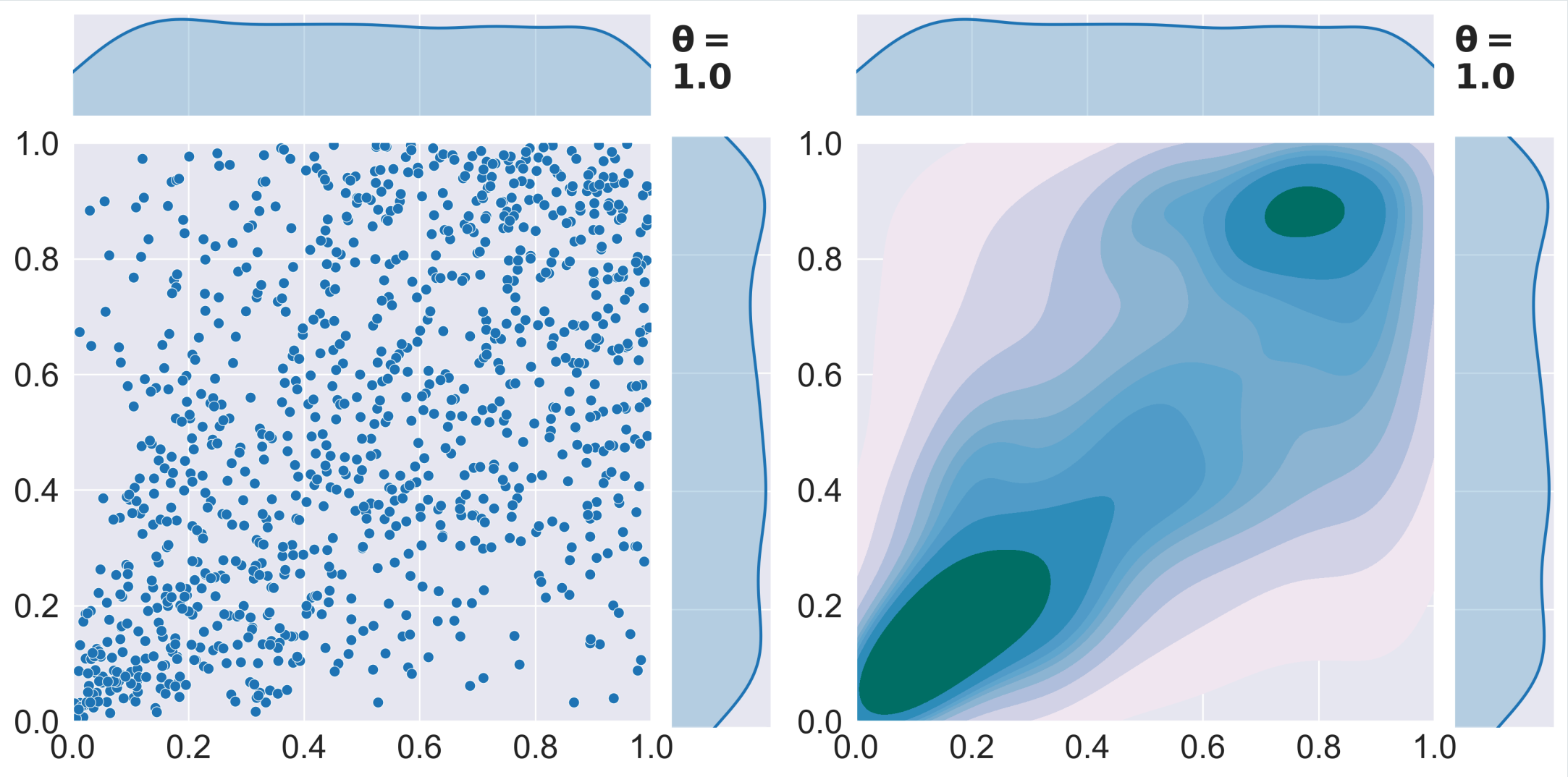
$$c^{-1}_{\theta,u}(v) = \left(1 + u^{-\theta}\left(v^{-\frac{\theta}{1+\theta}} - 1\right)\right)^{-\frac{1}{\theta}}$$

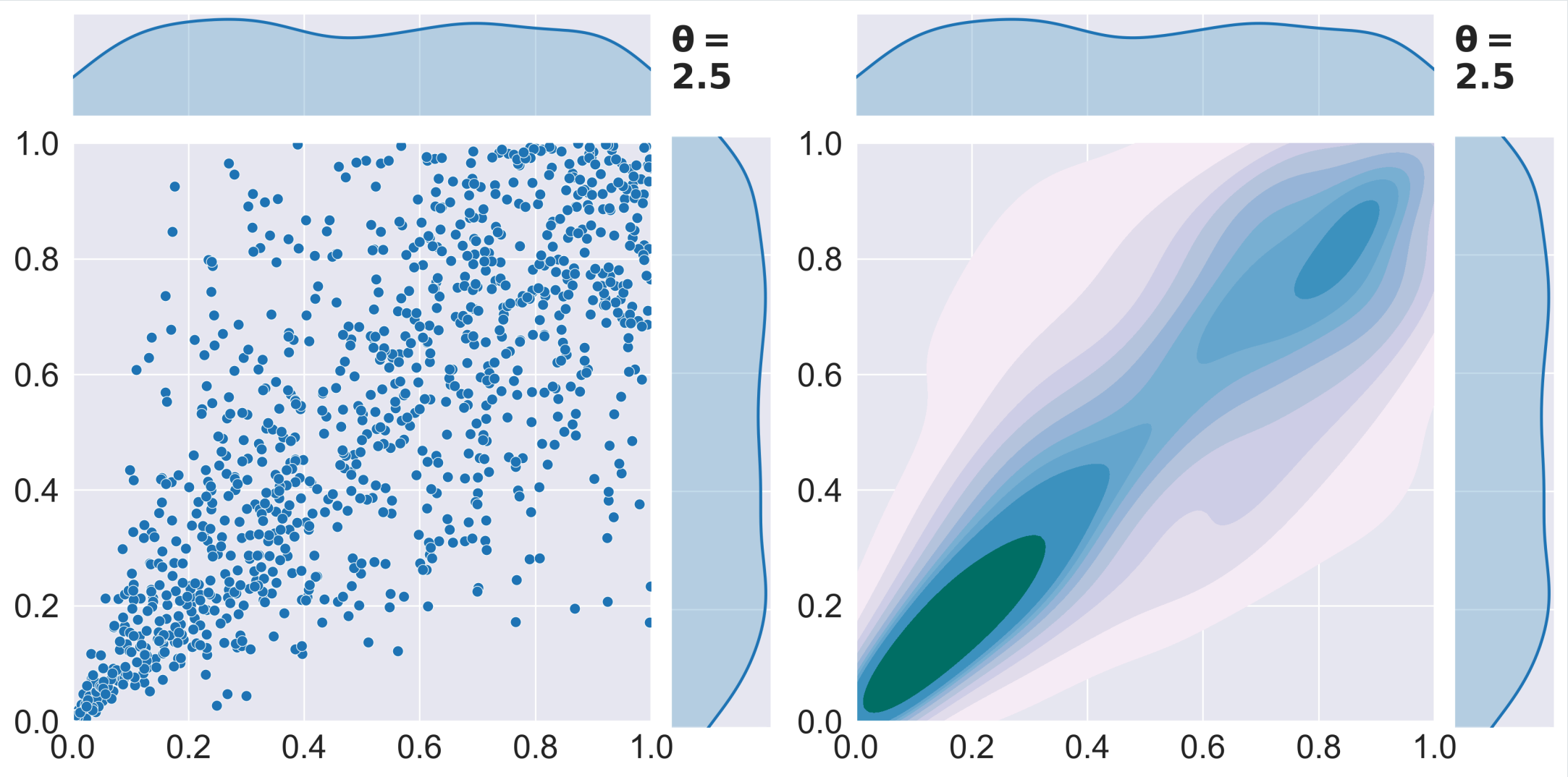
▷ Generate two independent uniform random variable simulations $(u_1, u_2) \in [0, 1]^2$

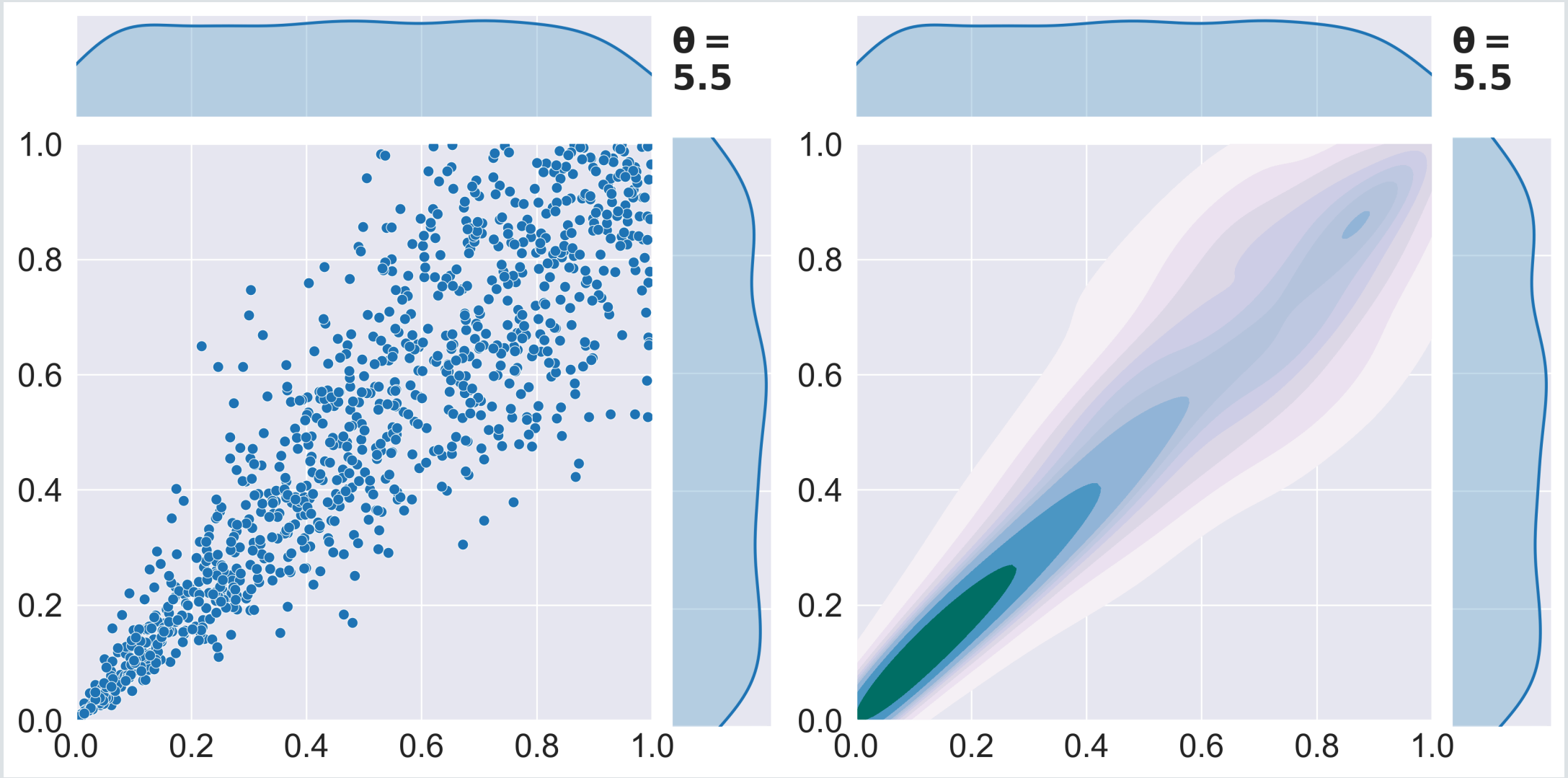
▷ Set $u = u_1$ and $v = \left(1 + u_1^{-\theta}\left(u_2^{-\frac{\theta}{1+\theta}} - 1\right)\right)^{-\frac{1}{\theta}}$

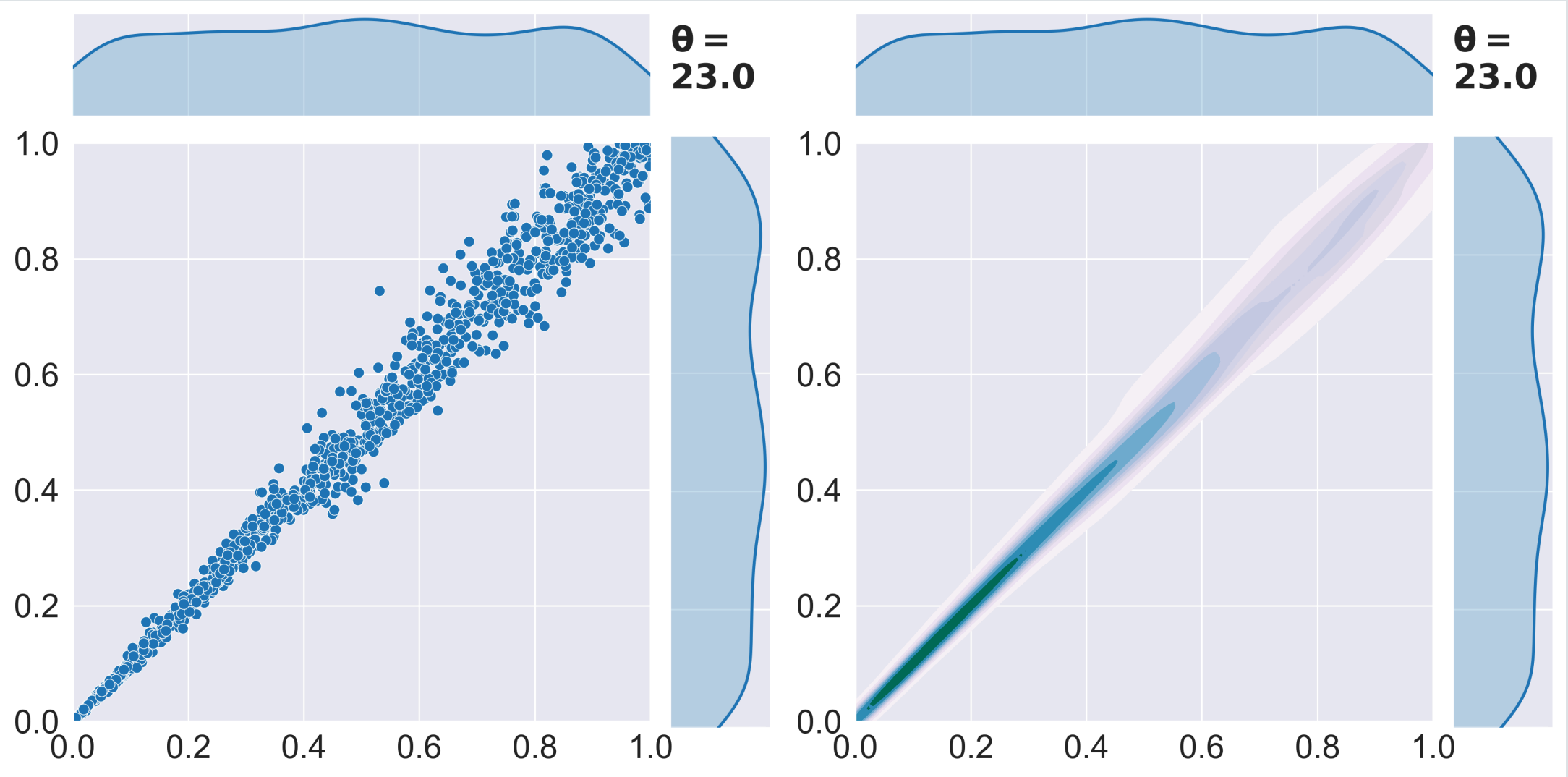

θ = 0.0
θ = 0.0
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


θ = 0.5
θ = 0.5
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0


θ = 1.0
θ = 1.0
0.0
0.2
0.4
0.6
0.8
1.0


θ = 2.5
θ = 2.5
0.0
0.2
0.4
0.6
0.8
1.0


θ = 5.5
θ = 5.5
0.0
0.2
0.4
0.6
0.8
1.0


θ = 23.0
θ = 23.0
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

Gaussian copula

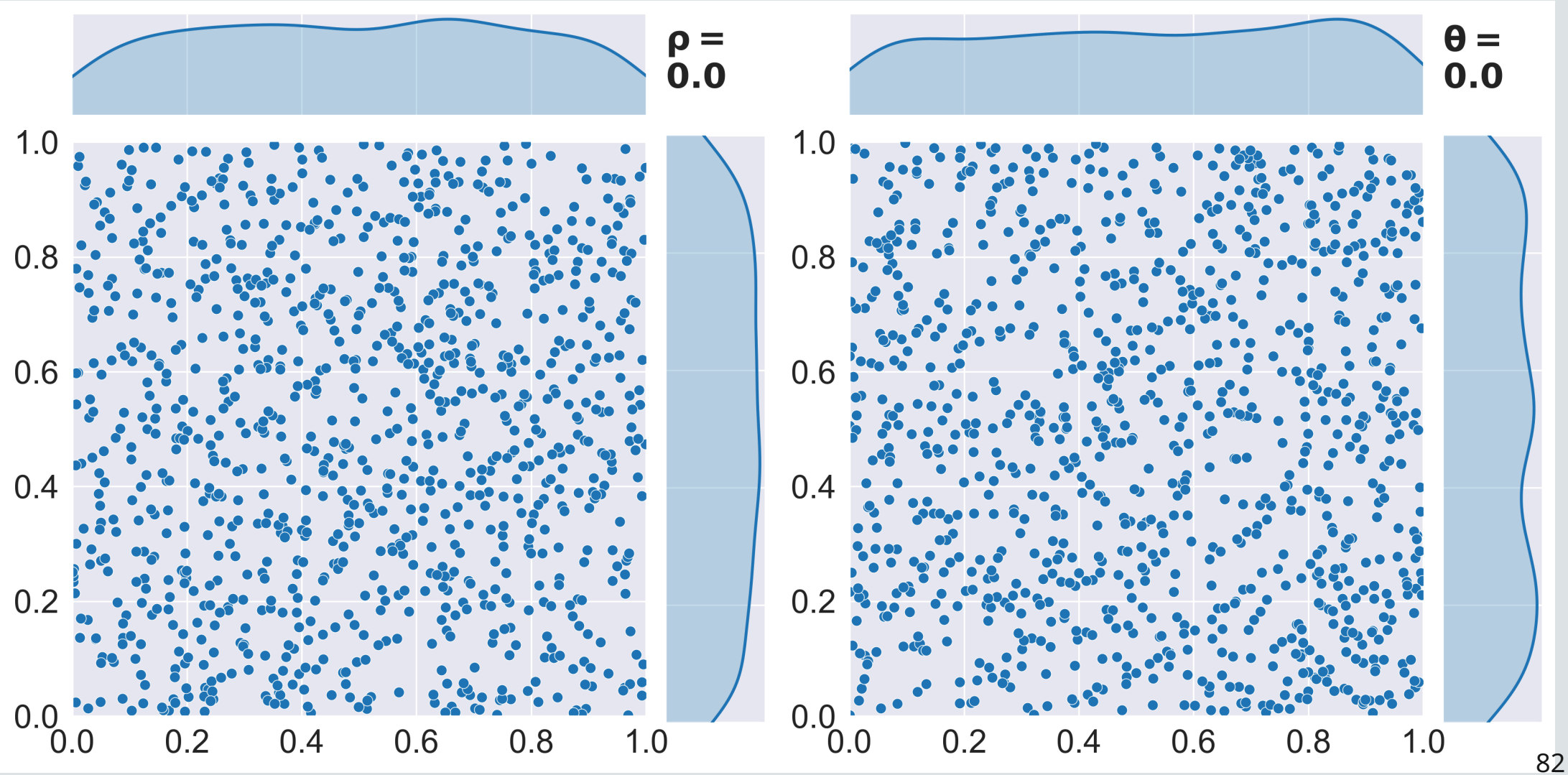
The bivariate Gaussian copula is defined by

$$\mathcal{C}_\rho(u, v) = \Phi_\rho\left(\Phi^{-1}(u), \Phi^{-1}(v)\right)$$

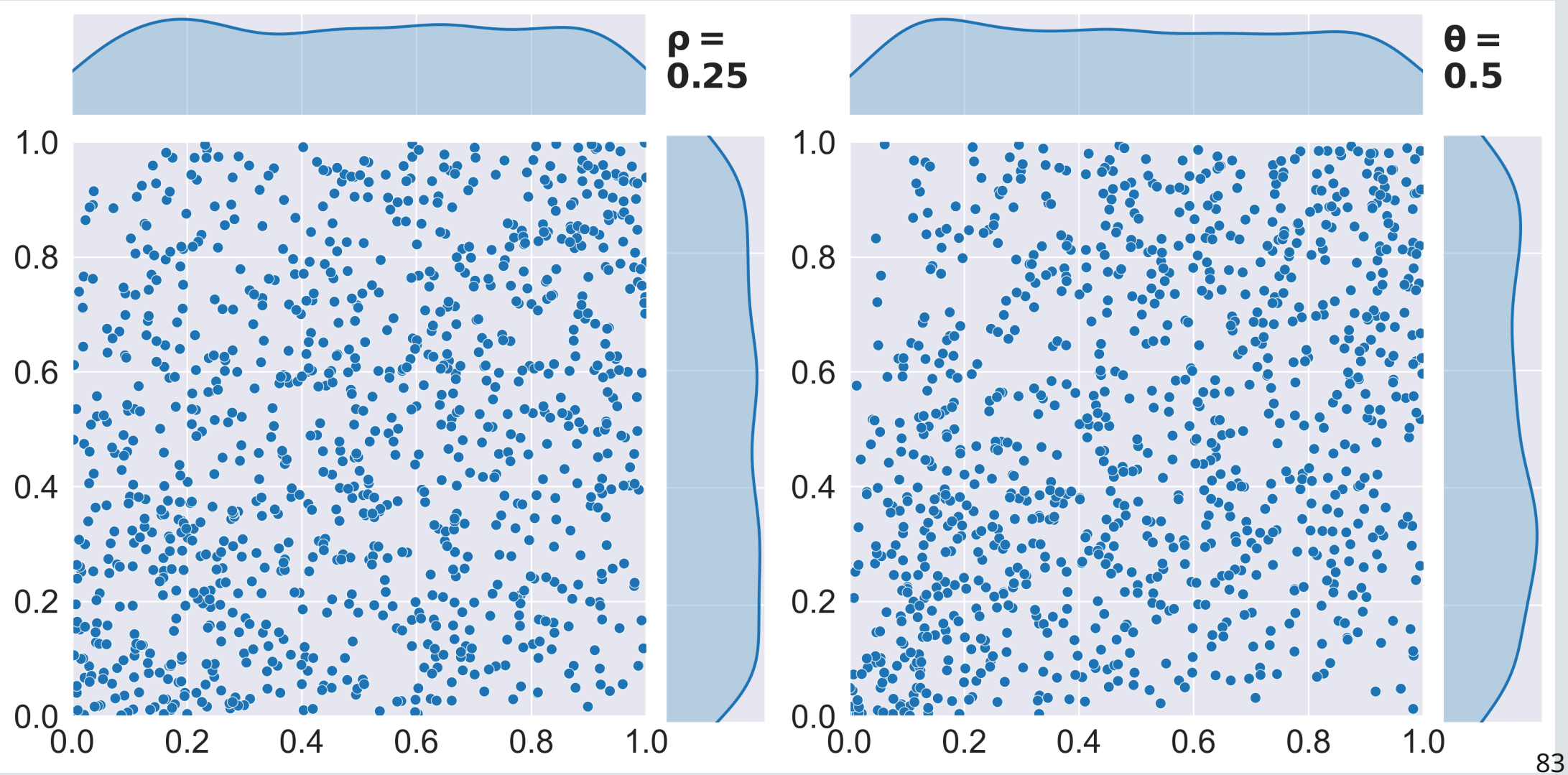
where Φ is the standard Gaussian CDF and Φ_ρ is the standard bivariate Gaussian CDF with correlation ρ. Simulating from the Gaussian copula follows the same steps as the NORTA and Iman-Conover methods in the Gaussian-to-uniform case:

▷ Simulate two Gaussian variables G_1 and G_2 with correlation ρ.
▷ Define $U_1 := \Phi(G_1)$ and $U_2 := \Phi(G_2)$ where Φ is the standard Gaussian CDF.

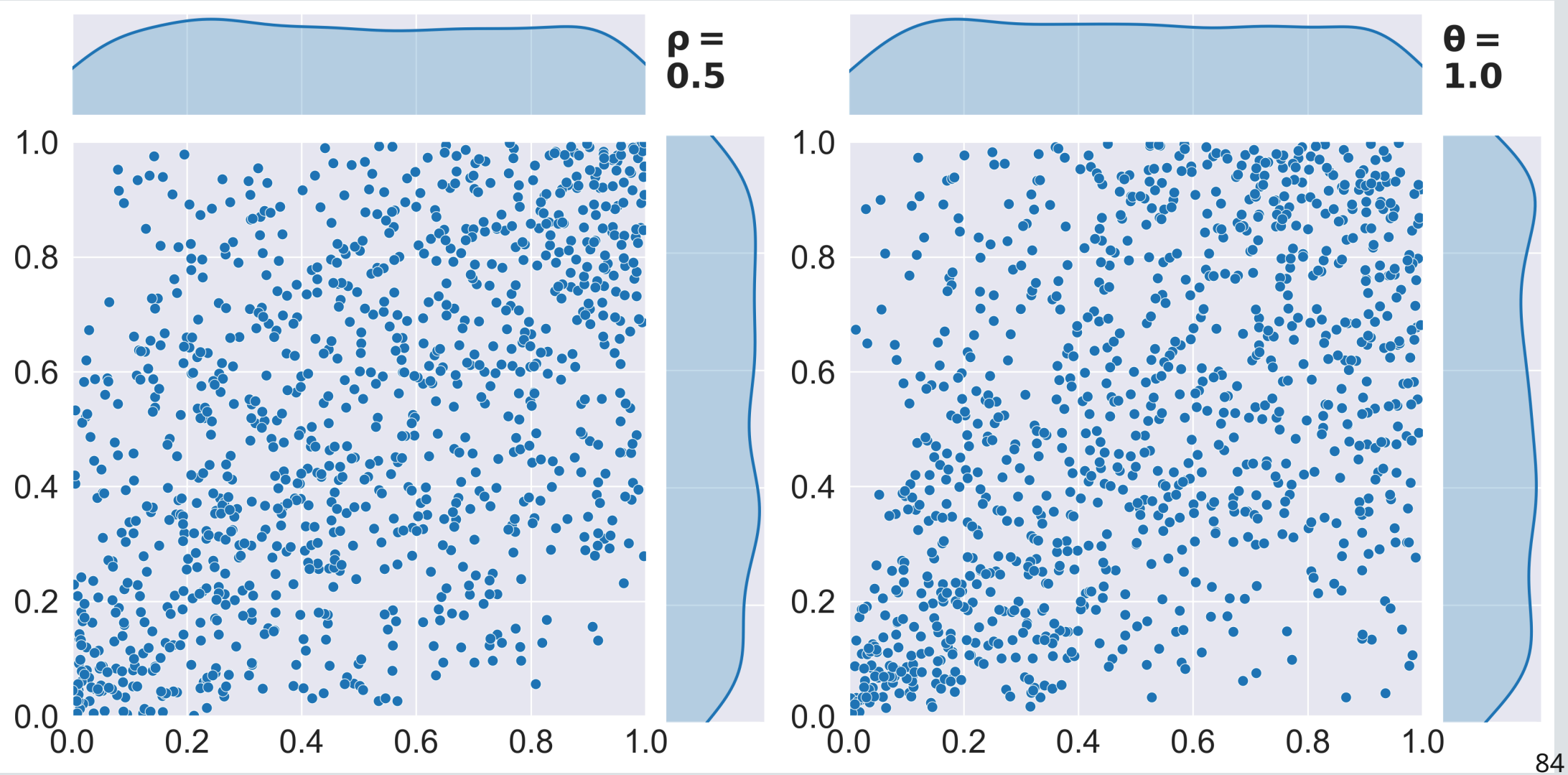
Gaussian copula vs. Clayton copula



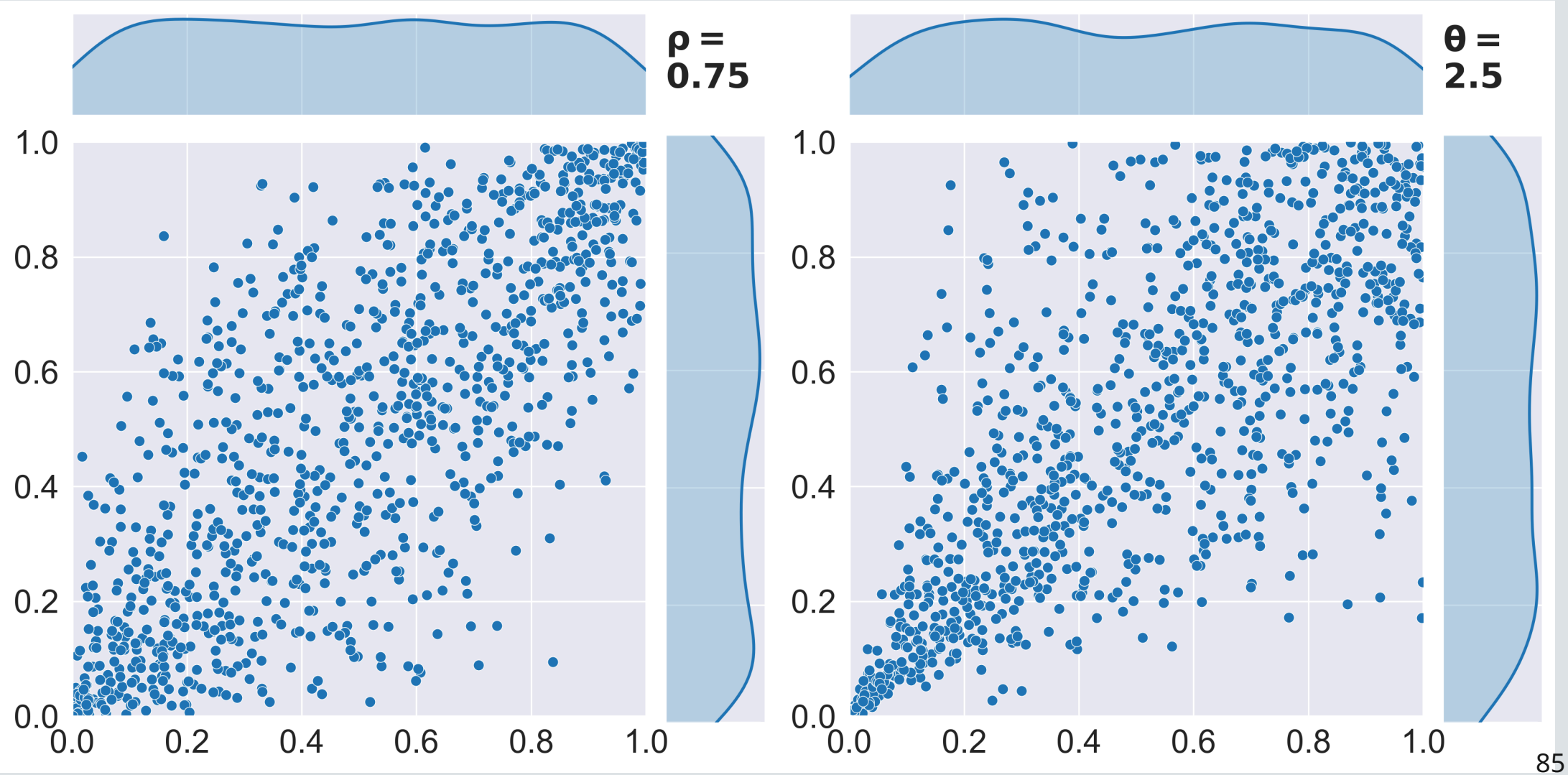
Gaussian copula vs. Clayton copula



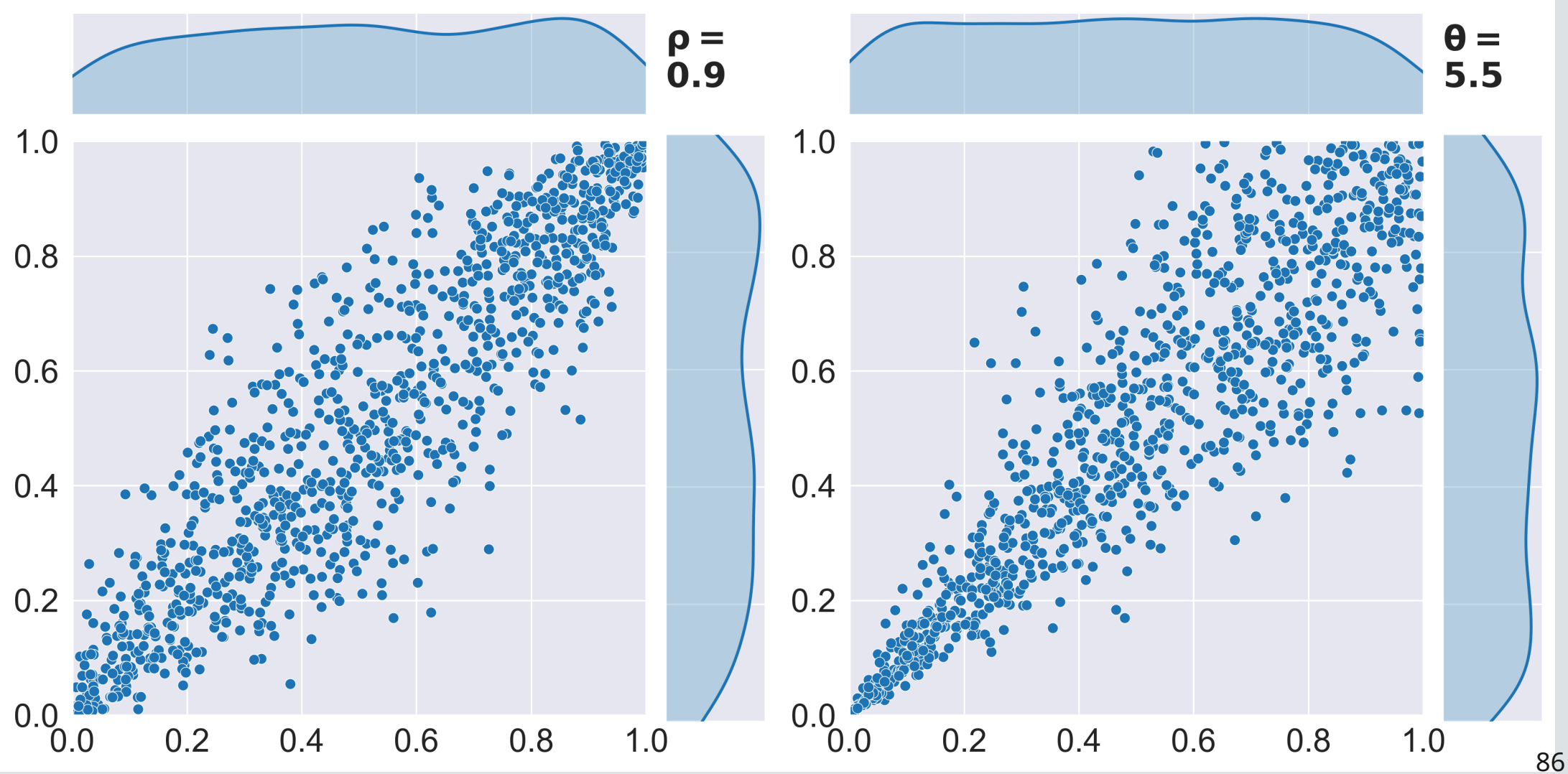
Gaussian copula vs. Clayton copula



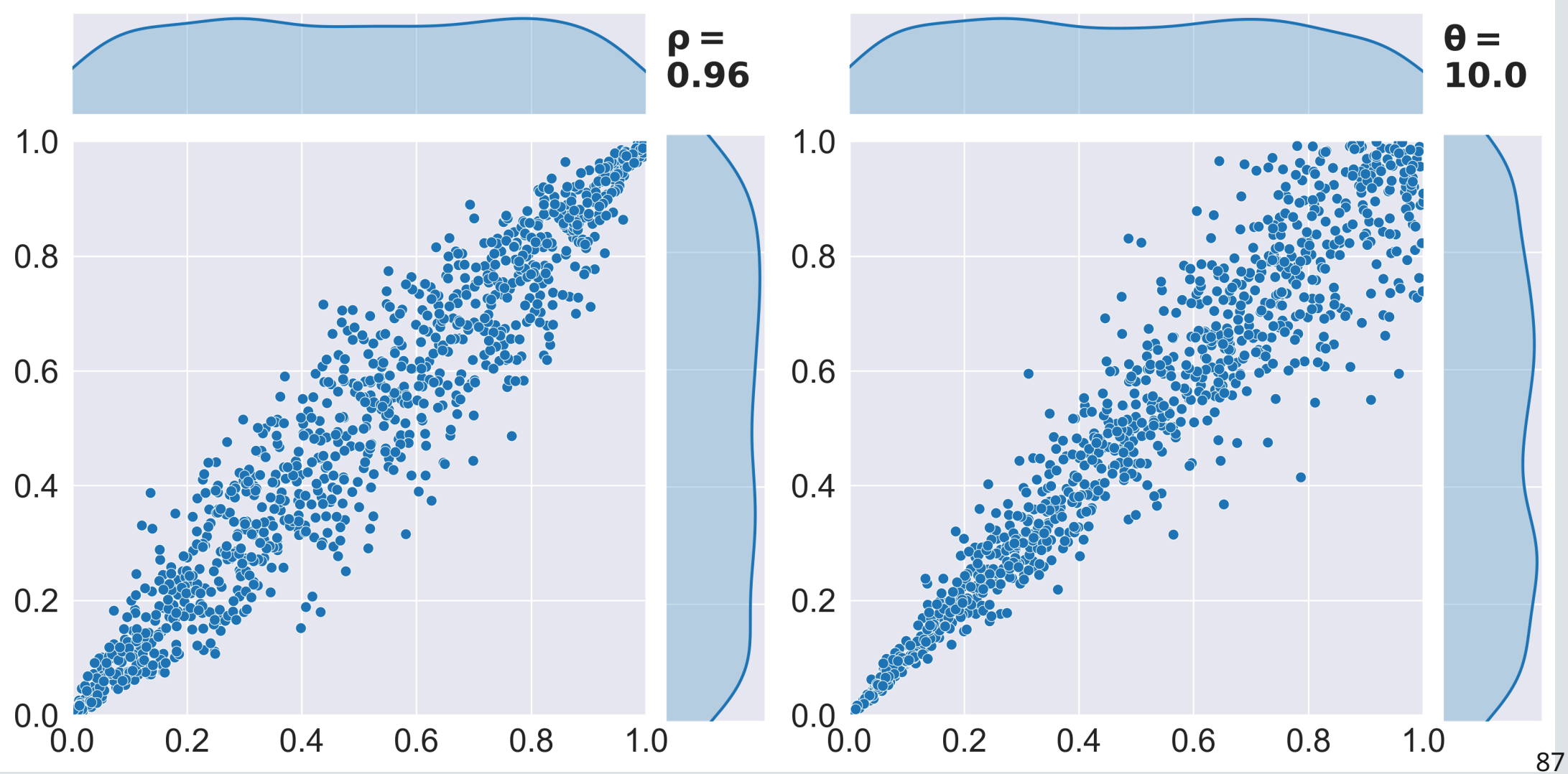
Gaussian copula vs. Clayton copula



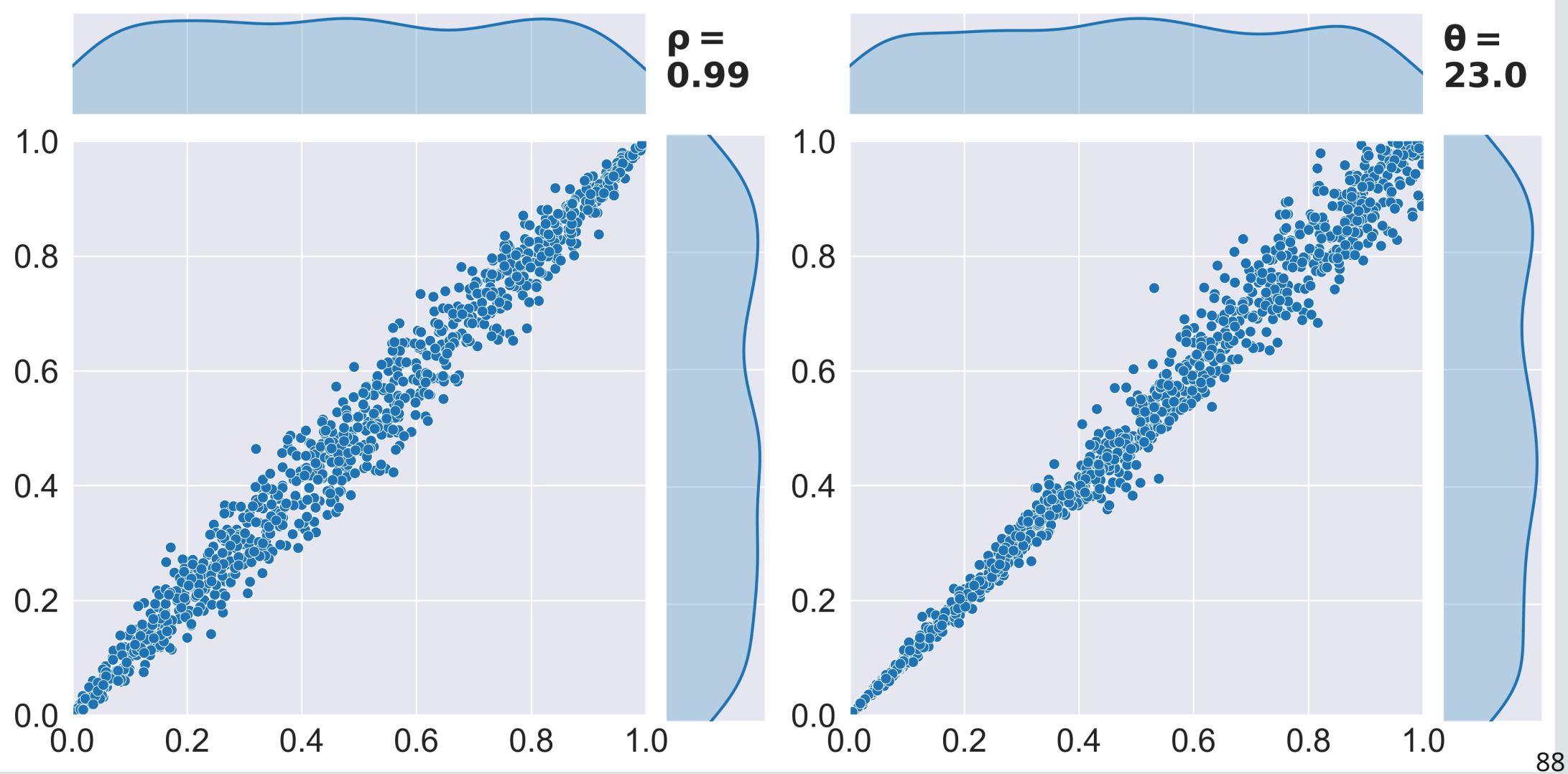
Gaussian copula vs. Clayton copula



Gaussian copula vs. Clayton copula



Gaussian copula vs. Clayton copula



Time series with lower tail dependence

As an example of application in finance, we simulate two time series with returns following a t distribution, and with dependence defined by a Clayton copula.
Let $S_1 = (S_{1,n})_{n=0,1,\ldots,N}$ and $S_2 = (S_{2,n})_{n=0,1,\ldots,N}$ be two time series. On each time period, the rates of return $R_{1,n}$ and $R_{2,n}$ are defined by

$$R_{1,n} = \frac{S_{1,n+1} - S_{1,n}}{S_{1,n}} \qquad R_{2,n} = \frac{S_{2,n+1} - S_{2,n}}{S_{2,n}}$$

Using these returns, the values of the time series can be expressed as

$$S_{1,n} = S_{1,0} \times (1 + R_{1,0}) \times \cdots \times (1 + R_{1,n-1})$$
$$S_{2,n} = S_{2,0} \times (1 + R_{2,0}) \times \cdots \times (1 + R_{2,n-1})$$

Time series with lower tail dependence

We assume that the returns $R_{1,n}$ and $R_{2,n}$ are t distributed with dependence defined by a Clayton copula. The time series S_1 and S_2 can be simulated as follows:

▷ Simulate $2N$ **independent** uniform random variables U_n and V_n, $n = 1, 2, \ldots, N$

▷ Use **conditional sampling** to introduce dependence between the two uniform series according to the Clayton copula with parameter θ. Define

$$U_{1,n} := U_n \qquad U_{2,n} = \left(1 + U_n^{-\theta}\left(V_n^{-\frac{\theta}{1+\theta}} - 1\right)\right)^{-\frac{1}{\theta}}$$

Time series with lower tail dependence

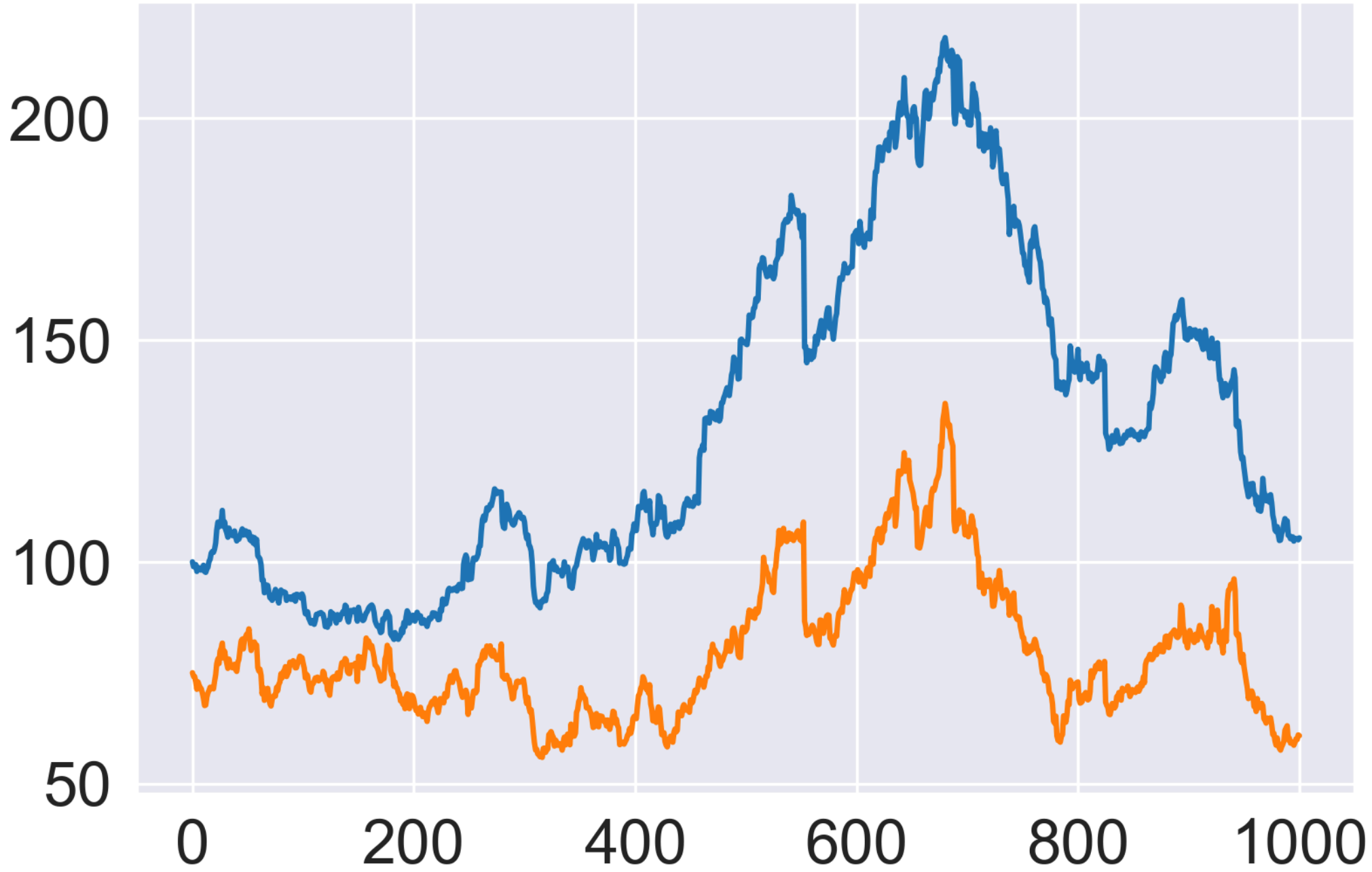
▷ Use the **inversion method** with the Student t CDF F_ν to obtain the returns R_1 and R_2:

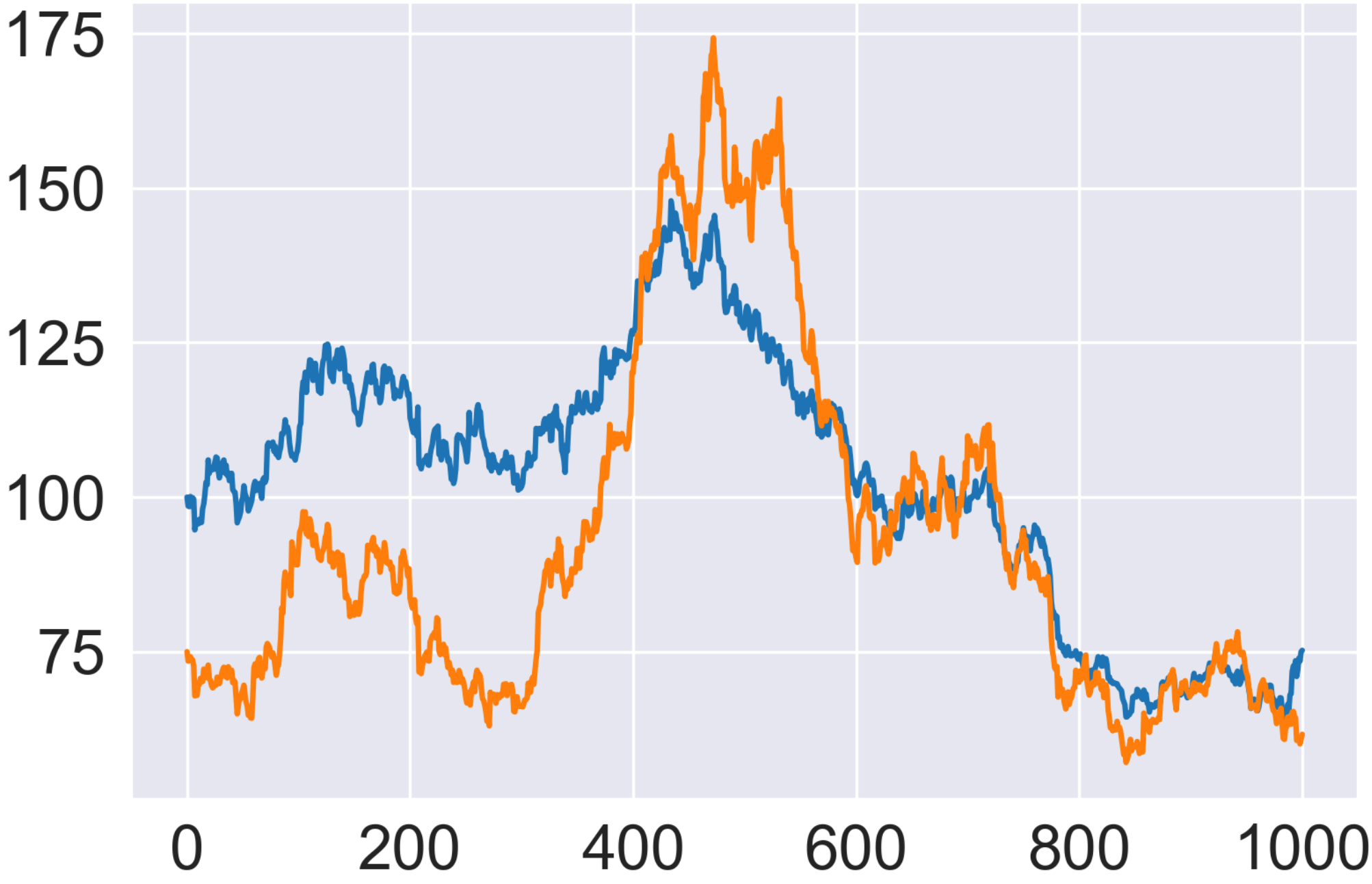
$$R_{1,n} := F_\nu^{-1}(U_{1,n}) \qquad R_{2,n} := F_\nu^{-1}(U_{2,n})$$

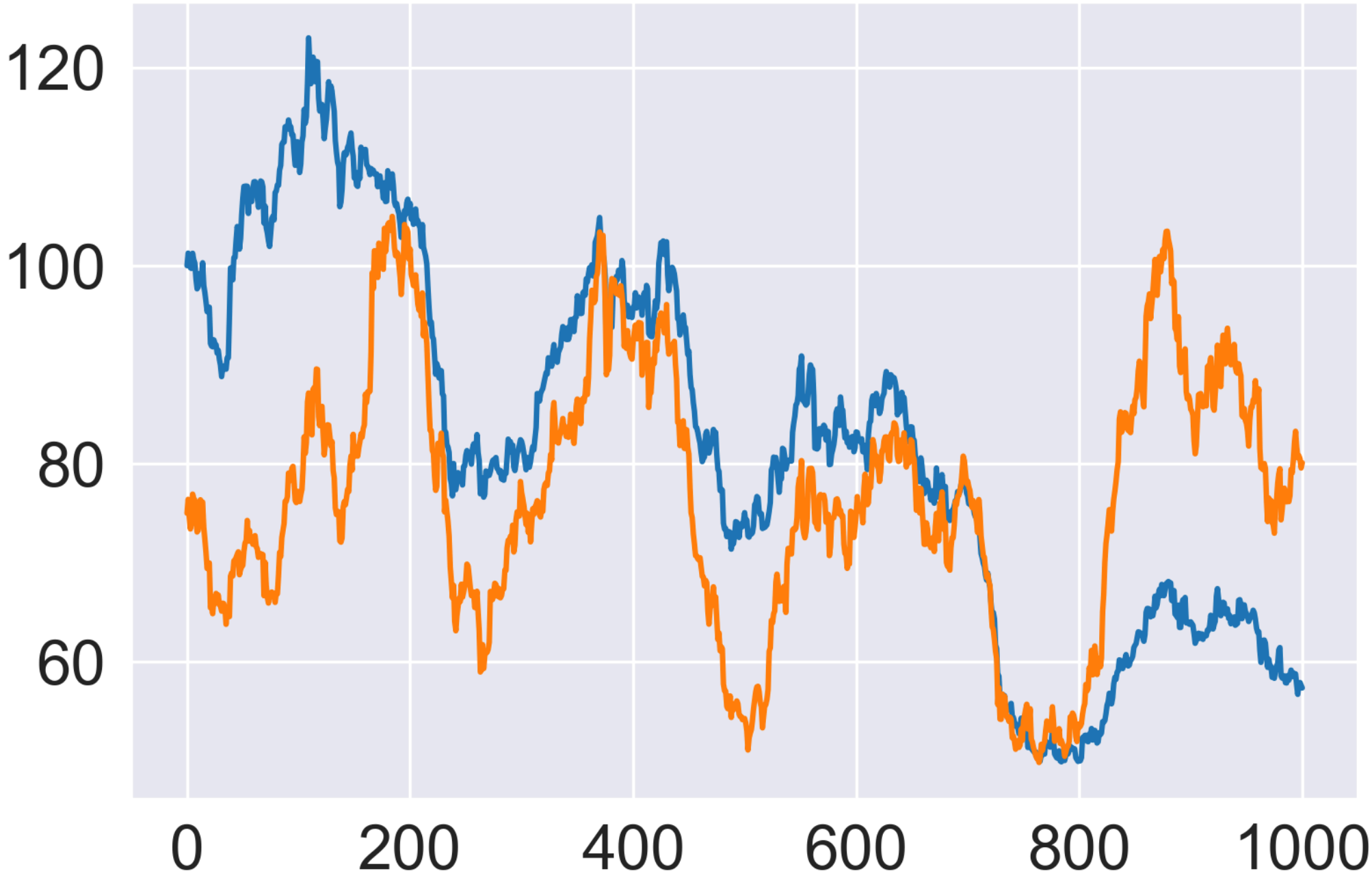
▷ Use these returns to compute the simulation of the time series S_1 and S_2 starting from the initial values $S_{1,0}$ and $S_{2,0}$:

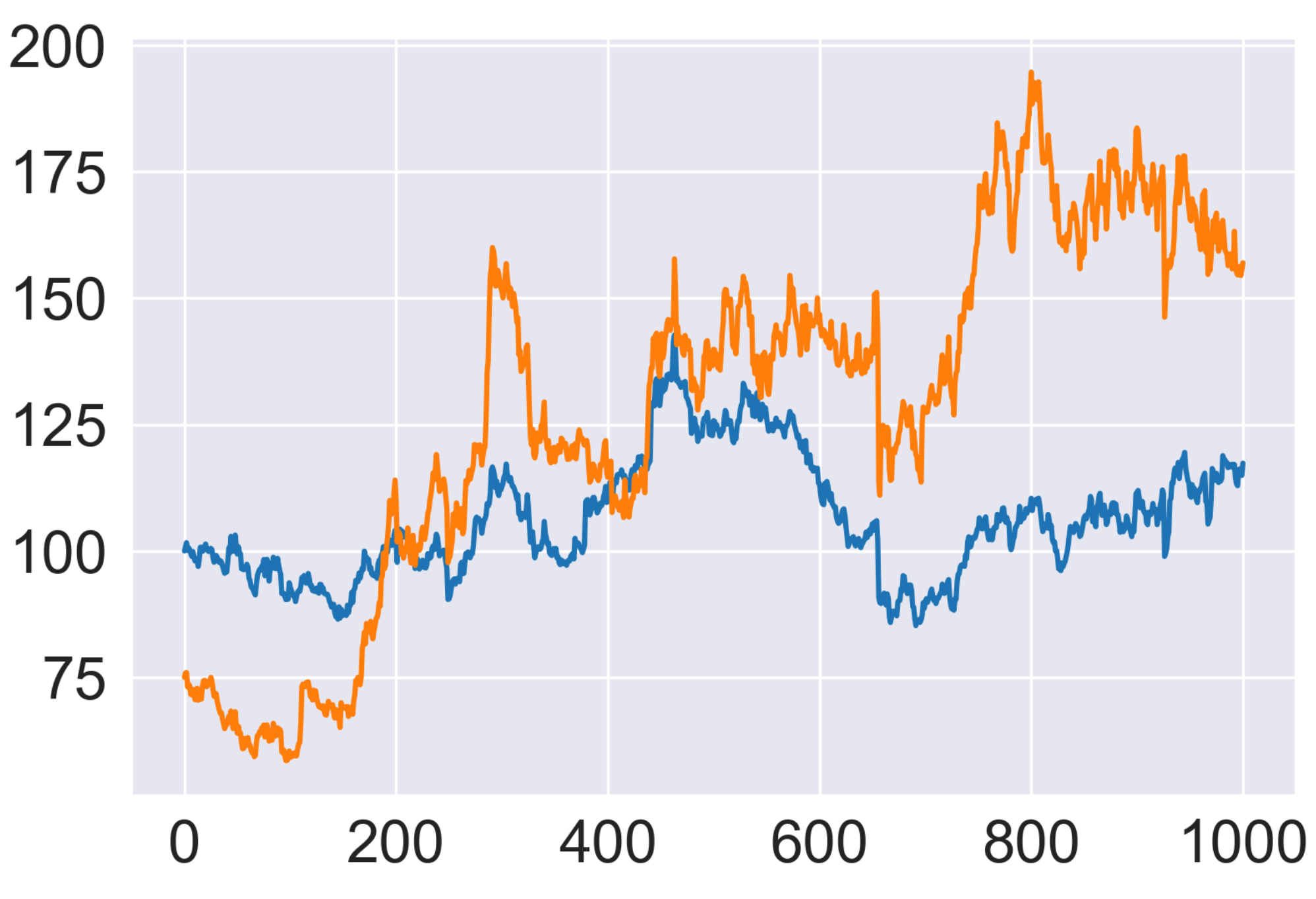
$$S_{1,n} = S_{1,0} \times (1 + R_{1,0}) \times \cdots \times (1 + R_{1,n-1})$$
$$S_{2,n} = S_{2,0} \times (1 + R_{2,0}) \times \cdots \times (1 + R_{2,n-1})$$

```
import numpy as np
import seaborn as sns
from scipy.stats import uniform, t
theta = 2.5 # Clayton copula parameter
M = 1000 # number of simulations
U = uniform.rvs(size=M)
V = uniform.rvs(size=M)
U1 = U
U2 = ( 1+U**(-theta) * (V**(-theta/(1+theta)) -1) )**(-1/theta)
X1 = t.ppf(U1,df=4,loc=0.0,scale=0.2/np.sqrt(365)) # daily volatility ~ 1%
X2 = t.ppf(U2,df=4,loc=0.0,scale=0.3/np.sqrt(365)) # daily volatility ~ 1.5%
S1 = np.zeros(M+1)
S2 = np.zeros(M+1)
S1[0] = 100 # initial price of stock S1
S2[0] = 75  # initial price of stock S2
for i in np.arange(M):
    S1[i+1] = S1[i]*(1+X1[i])
    S2[i+1] = S2[i]*(1+X2[i])
fig, ax = plt.subplots()
sns.lineplot(x=np.arange(M+1),y=S1,ax=ax)
sns.lineplot(x=np.arange(M+1),y=S2,ax=ax)
plt.show()
```


200
150
100
50
0
200
400
600
800
1000


175
150
125
100
75
0
200
400
600
800
1000


120
100
80
60
0
200
400
600
800
1000


200
175
150
125
100
75
0
200
400
600
800
1000

Other copulas

There exist many other copulas (Gumbel copula, Frank copula, Student's t copula) and all of them can be defined in the general multivariate setting (see also *Vine copulas*).

More information is available in the following textbooks:

[1] U. Cherubini, E. Luciano, and W. Vecchiato. *Copula methods in finance*. John Wiley & Sons, 2004.

[2] R. Nelsen. *An introduction to copulas*. Springer, 2007.